

Prepared By: Hatim Elbakkali

Topic: Course JAVASCRIPT

Year: 2026

Table of Contents

>Algorithm:

1-Data Type:	7
1-2-How do you know the data type?	8
2-Variable:	9
2-1-What different (Var, Let, Const):	9
2-2-Namming Variable:	11
3-Template Literals:	12
4-Arithmetic Operator:	13
4-1-Post and Pre-Increment :	14
4-2-Post and Pre-Decrement :	14
4-3-Unary Plus And Negation Operators:	15
4-4-Type Coercion:	16
4-5-Assignment Operators:	17
5-Methods Number:	18
5-1-Methode Math:	19
6-Methode String:	20
7-Comparison:	24
8-Logical Operators:	25

9-Condition:	26
9-1-Nested Condition:	27
9-2-Conditional Ternary Operator:	28
9-3- CNullish Coalescing Operator And Logical Or:	29
9-4-Switch Statement:	30
10-Loop For:	31
10-1-Loopin On Sequences:	32
10-2-Nested loop for:	33
10-3-Loop Control - Break, Continue, Label:	34
11-Loop While:	35
11-1-Loop Do While:	36
12-Function:	37
12-1-Built in function VS user defined function:	37
12-2-Function Simple and Parameter:	37
12-3-Anonymous Function :	41
12-4-Arrow Function :	42
13- Higher order Function:	43
13-1- Map:	43

13-2-ForEach:	44
13-3- Filter:	45
13-4-Reduce:	
>Data Structure:	46
1- Array:	47
1-1-Nested Array:	48
1-2-Index and Length With Array:	49
1-3- Methode Array:	50
1-3-1- Add And Remove From Array:	50
1-3-2-Searching Array:	51
1-3-3-Sorting Array:	52
1-3-4-Array Operations (Slice, Splice, Concat, Join) :	52
2- Object:	54
2-1- Dot Notation VS Bracket Notation :	55
2-2- Nested Object:	56
2-3- Methods Object:	57
3-Map And Set:	
	59

>Advanced Concepts:

1-DOM (Document Object Model):	60
1-1-DOM Selectors:	60
1-2-DOM Deal With Children's :	62
1-3-DOM Traversing:	64
1-4-Get & Set Elements Content And Attributes:	65
1-5-Class List:	67
1-6-Create And Append Elements:	69
1-7-DOM CSS:	72
1-8-DOM Event and AddEventListener:	74
1-8-1-Event Mouse:	74
1-8-2-Form Event:	75
1-8-3-Window Event:	75
2-BOM(Browser Object Model):	76
2-1-SetTimeout and clearTimeout:	76
2-2-setInterval and clearInterval:	78
2-3-Window Methods:	79
3-Local Storage:	81
4-Session Storage:	83

5-Destructuring:	85
6-Regular Expression(Regex):	88
6-1-Rules Regex:	88
6-1-1-Characters:	89
6-1-2-Quantifiers:	90
6-1-3-Position:	91
6-1-4-Groups And Ranges:	92
7-Date, Generator, Modules:	93
7-1-Date:	93
7-1-1-get Date And Time:	93
7-1-2-Set Date And Time:	96
7-2-Generators:	98
7-2-1-Generator Delegate:	99
7-3-Modules:	100
7-3-1-Export:	100
7-3-2-Import:	101
7-3-3-Import All:	101
1-JSON (JavaScript Object Notation):	102
2-Object VS JSON:	103

3-Synchronous VS ASynchronous:	105
4-AJAX:	106
5-Promise:	110
5-1-then, catch, finally:	111
5-2-Promise.all,Promise.allSettled, Promise.race :	112
6-Fetch API:	114
7-Async/Await, Try:	116

JAVASCRIPT

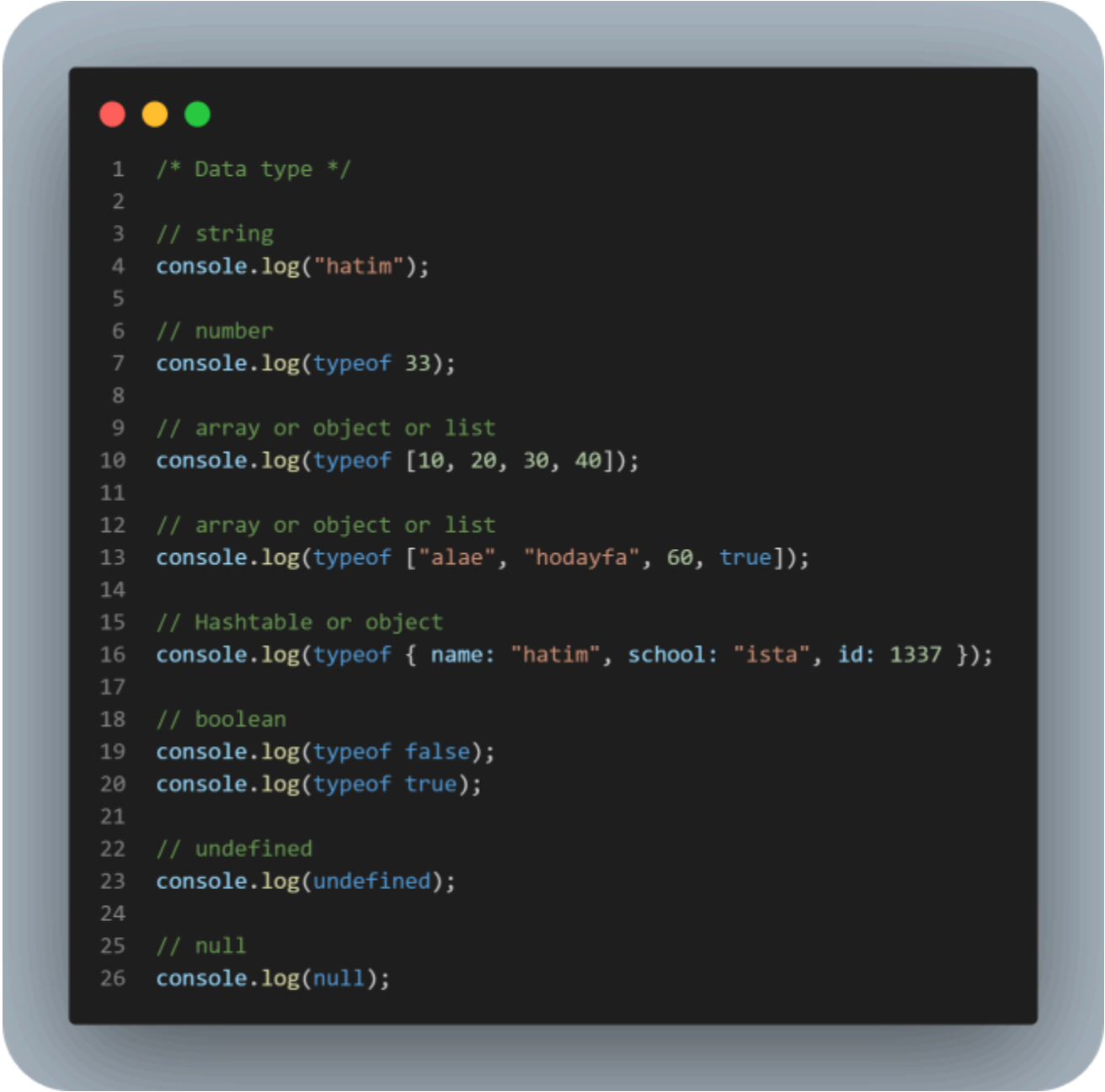


is a programming language used to make web pages interactive, such as buttons, forms, and animations on a page.

>Algorithm:

1-Data Type:

The type of values that can be stored within variables, which determines how the program handles these values.



```
1  /* Data type */
2
3  // string
4  console.log("hatim");
5
6  // number
7  console.log(typeof 33);
8
9  // array or object or list
10 console.log(typeof [10, 20, 30, 40]);
11
12 // array or object or list
13 console.log(typeof ["alae", "hodayfa", 60, true]);
14
15 // Hashtable or object
16 console.log(typeof { name: "hatim", school: "ista", id: 1337 });
17
18 // boolean
19 console.log(typeof false);
20 console.log(typeof true);
21
22 // undefined
23 console.log(undefined);
24
25 // null
26 console.log(null);
```

1-2-How do you know the data type?

Use:



```
1 // number
2 console.log(typeof 33);
```

2-Variable:

A space in memory that we use to store data (values such as a number, String, etc.) for later use in the program Like a box where we put information.

2-1-How do we create a Variable?

Key(Var, Let, Const) + Name Variable = "hatim"

2-1-What different (Var, Let, Const):

var: It's the old way of defining variables in JavaScript.

Properties:

- It can be redefined (re-declare: Redefine the variable).
- Its value can be changed (re-assign: Change value only).
- Block Scope doesn't respect well

let: A modern way to define variables.

Properties:

- It cannot be redefined within the same scope.
- Its value can be changed.
- Blockscope respects.

Const: Used to define a fixed value that does not change.

Properties:

- It cannot be redefined.
- Its value cannot be changed.
- Blockscope respects

Example:



```
1 // var يسمح بالاثنين
2 var a = 10;
3 var a = 20; // re-declare ✓
4 a = 30;      // re-assign ✓
5
6 // let يسمح فقط بـ re-assign
7 let b = 10;
8 b = 20;      // ✓
9 // let b = 30; خطأ ✗
10
11 // const لا يسمح بأي تغيير
12 const c = 10;
13 // c = 20; ✗
14 // const c = 30; ✗
```

2-2-Namming Variable:

The rules we use to choose a variable name.

Variable naming rules:

use:

- It must start with a letter or _ or \$.
- use CamelCase or UpperCase.

Not Use:

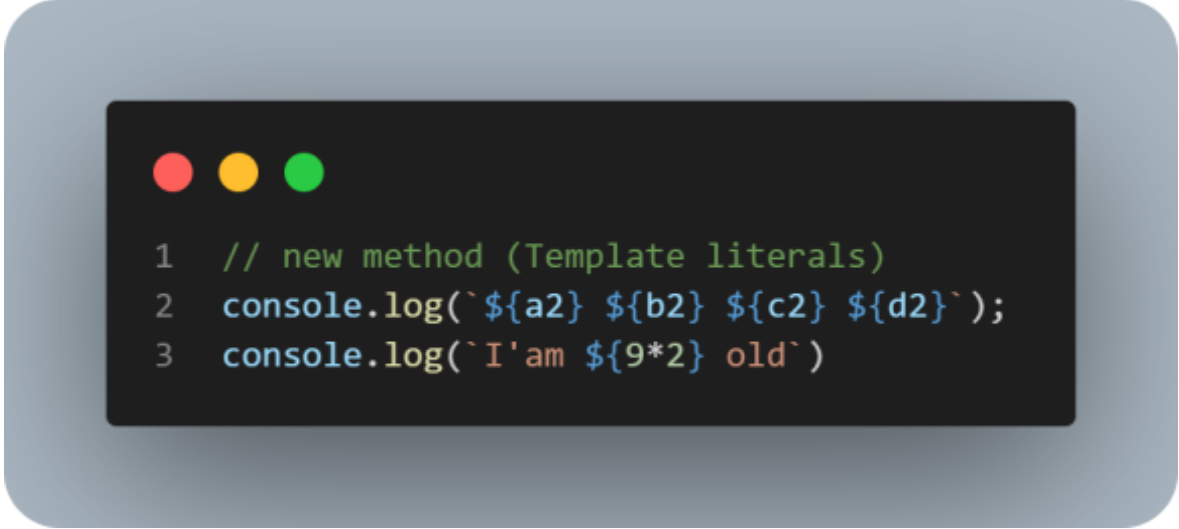
- It cannot start with a number.
- Distances are not allowed.
- The name must be clear.

3-Template Literals:

It is a modern way to write texts (Strings) with ease and flexibility. We use Backticks(`) instead of (" ") or (' ').

Why do we use it?

- Easily merge variables within text.
- Write multi-line text.
- Clearer and easier code.

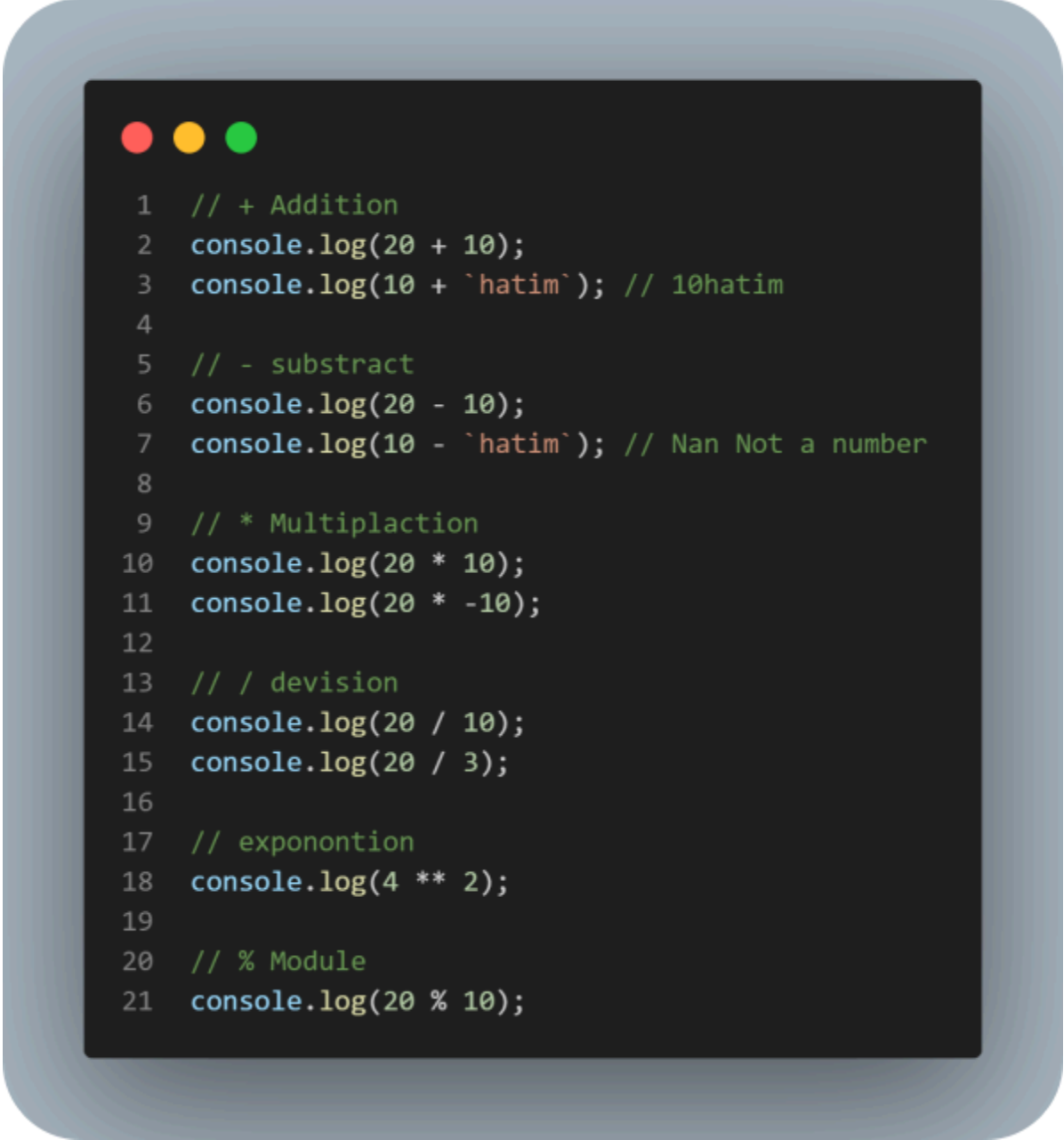


```
1 // new method (Template literals)
2 console.log(`${a2} ${b2} ${c2} ${d2}`);
3 console.log(`I'am ${9*2} old`)
```

4-Arithmetic Operator:

These are operations used to perform calculations on numbers.

Example:



```
1  // + Addition
2  console.log(20 + 10);
3  console.log(10 + `hatim`); // 10hatim
4
5  // - subtract
6  console.log(20 - 10);
7  console.log(10 - `hatim`); // Nan Not a number
8
9  // * Multiplaction
10 console.log(20 * 10);
11 console.log(20 * -10);
12
13 // / devision
14 console.log(20 / 10);
15 console.log(20 / 3);
16
17 // exponontion
18 console.log(4 ** 2);
19
20 // % Module
21 console.log(20 % 10);
```

4-1-Post and Pre-Increment :

is used to increase the value of a variable by 1, but the difference is when to expect the operation.



```
1 // Post-increment: 1 increases but only after it returns the value
2 let x = 5;
3 console.log(x++);
4 console.log(x);
5
6 // Pre-increment: 1 increases but it returns the value Now
7 let y = 5;
8 console.log(++y);
```

4-2-Post and Pre-Decrement :

Decrement(--) is a process used to decrement the value of a variable by 1.



```
1 // Post-decrement: 1 decreases but only after it returns the value
2 let x = 5;
3 console.log(x--);
4 console.log(x);
5
6 // Pre-decrement: 1 decreases but it returns the value Now
7 let y = 5;
8 console.log(--y);
```


4-3-Unary Plus And Negation Operators:

Unary Plus: It is a single operator used to convert a value to a number (Number) without changing its sign.

Unary Negation: It is a single operator used to convert a value to a number and then change its sign (positive ↔ negative).



```
1  // Unary Plus
2  console.log(+ "90"); //90
3  console.log(+ "-90"); //-90
4  console.log(+ "hatim"); //NaN
5  console.log(+ true); // 1
6  console.log(+ false); //0
7  console.log(+ null); //0
8  console.log(+ 0xff); //hex
9
10 // Unary Negation
11 console.log(- "90"); //-90
12 console.log(- "-90"); //90
13 console.log(- "hatim"); //NaN
14 console.log(- true); //-1
15 console.log(- false); //-0
16 console.log(- null); //-0
17 console.log(- 0xff); //hex
```

4-4-Type Coercion:

automatically converts a data type from one type to another while performing operations, without you writing that conversion.

Example:



```
1  let a = "36";
2  let b = 6;
3  let c = true;
4  let d = false;
5  console.log(a / b); // 6
6  console.log(a - b); // 30
7  console.log(a * b); // 216
8  console.log(b + c); // 7
9  console.log(b + d); // 6
10 console.log(+a + b + c); // 43
```

4-5-Assignment Operators:

These are operations that are used to give a value to a variable or modify its value in short.

Example:



```
1  let h = 10;  
2  h += 100;//110  
3  h -= 10;//100  
4  h /= 10;//10  
5  
6  console.log(h);
```

5-Methods Number:

are functions (functions) used to manipulate numbers: transform them, format them, or analyze them.

```
1
2 // toString()
3 // تعريف: تحويل الرقم إلى نص (String)
4 console.log((100).toString()) // "100"
5
6 // toFixed()
7 // تعريف: تحديد عدد الأرقام بعد الفاصلة مع التقريب
8 console.log(100.666666)
9 console.log(10.6666666.toFixed(2)) // "10.67"
10
11 // parseFloat()
12 // تعريف: تحويل النص إلى عدد عشري (Float)
13 console.log(parseFloat("67.8989")) // 67.8989
14
15 // parseInt()
16 // تعريف: تحويل النص أو الرقم إلى عدد صحيح (Integer)
17 console.log(parseInt(76.09090)) // 76
18
19 // Number.isInteger()
20 // تعريف: التحقق هل القيمة عدد صحيح (بدون فاصلة)
21 console.log(Number.isInteger(66)) // true
22 console.log(Number.isInteger("66")) // false
23
24 // Number.isNaN()
25 // NaN (Not a Number) تعريف: التحقق هل القيمة هي
26 console.log(Number.isNaN(66)) // false
27 console.log(Number.isNaN("66")) // false
28 console.log(Number.isNaN("ha" / 90)) // true
```

5-1-Methode Math:

is a ready-made object (built-in) that contains a set of functions and constants that help you manage mathematical operations easily.

```
1 // Math.round() -> كيقرب الرقم لأقرب عدد صحيح
2 console.log(Math.round(99.2)); // 99
3 console.log(Math.round(99.7)); // 100
4
5 // Math.ceil() -> كيرفع الرقم دائماً للعدد الصحيح اللي فوق
6 console.log(Math.ceil(99.1)); // 100
7
8 // Math.floor() -> كينقص الكسر وكيهبط دائماً
9 console.log(Math.floor(99.9)); // 99
10
11 // Math.min() -> كيرجع أصغر رقم من بين مجموعة أرقام
12 console.log(Math.min(1090, 70, 80, 10)); // 10
13
14 // Math.max() -> كيرجع أكبر رقم من بين مجموعة أرقام
15 console.log(Math.max(1090, 70, 80, 10)); // 1090
16
17 // Math.pow(base, exponent) -> كيرفع الرقم لقوة معينة (power)
18 console.log(Math.pow(2, 8)); // 256 (2^8)
19
20 // Math.random() -> كيعطي رقم عشوائي بين 0 و 1
21 console.log(Math.random());
22
23 // Math.trunc() -> كيشيل الكسر وكيخلي غير الجزء الصحيح (بلا تقريب)
24 console.log(Math.trunc(45.9)); // 45
```

Note:

There are many methods to search for Math. I have put the most important ones methode Math.

6-Methode String:

They are ready-made functions in JavaScript that are used to manipulate texts, such as modifying them, searching within them, dividing them, or changing their shape (such as uppercase and lowercase letters).

Example:



```
1 let the_name = " Elzero ";
2
3 // charAt() -> معين position كيعطيك الحرف فـ
4 console.log(the_name.charAt(2));
5
6 // length -> ديال النص length كيرجع عدد الحروف
7 console.log(the_name.length);
8
9 // trim() -> كيشيل المسافات من البداية والنهاية
10 console.log(the_name.trim());
11
12 // toUpperCase() -> كيقلب جميع الحروف لكبار
13 console.log(the_name.toUpperCase().trim());
14
15 // toLowerCase() -> كيقلب جميع الحروف لصغار
16 console.log(the_name.toLowerCase().trim());
17
18 // chaining methods -> مع بعض method كنجمعو أكثر من
19 console.log(the_name.trim().toUpperCase().charAt(2));
```



```
1 let a = `Elzero Web School`;
2
3 /*
4 indexOf()
5 → تبحث عن أول ظهور للنص داخل السلسلة
6 → (index) تُرجع موقعه
7 → index تعتمد على
8 */
9 console.log(a.indexOf(`Web`)); // 7
10
11 /*
12 lastIndexOf()
13 → تبحث عن آخر ظهور للنص داخل السلسلة
14 → (index) تُرجع آخر موقع
15 → index تعتمد على
16 */
17 console.log(a.lastIndexOf(`Web`)); // 7
18 console.log(a.lastIndexOf(`o`)); // 15
19
20 /*
21 slice(start, end)
22 → تقطع جزءاً من النص
23 → index تعتمد على
24 → end غير مشمولة
25 → تدعم القيم السالبة (من النهاية)
26 */
27 console.log(a.slice(7)); // Web School
28 console.log(a.slice(7, 10)); // Web
29 console.log(a.slice(-7, -10)); // ""
```



```
1  /*
2  repeat()
3  ➔ تكرر النص عددًا معينًا من المرات
4  ➔ index أو length لا تعتمد على
5  */
6  console.log(a.repeat(10)); // Elzero...(10 مرات)
7
8  /*
9  split()
10 ➔ تقسم النص إلى مصفوفة
11 ➔ (separator) حسب فاصل معين
12 ➔ index أو length لا تعتمد على
13 */
14 console.log(a.split(' ')); // ["Elzero", "Web", "School"]
15
16 /*
17 substring(start, end)
18 ➔ تقطع جزءًا من النص
19 ➔ index تعتمد على
20 ➔ لا تدعم القيم السالبة (تتحول إلى 0)
21 ➔ يتم تبديلهما end أكبر من start إذا كان
22 */
23 console.log(a.substring(2, 8)); // zero W
24 console.log(a.substring(8, 2)); // zero W
25 console.log(a.substring(-10, 7)); // Elzero
26
27 /*
28 substring مع length
29 ➔ لتحديد المواقع داخل النص length يتم استخدام
30 ➔ index و length دمج بين
31 */
32 console.log(a.substring(a.length - 5, a.length - 3)); // ho
```




```
1  /*
2  includes()
3  → تتحقق من وجود نص داخل السلسلة
4  → true أو false تُرجع
5  → (index) تعتمد على البحث
6  */
7  console.log(a.includes(`elzero`)); // false
8  console.log(a.includes(`Elzero`)); // true
9
10 /*
11 startsWith()
12 → تتحقق هل النص يبدأ بقيمة معينة
13 → index تعتمد على
14 */
15 console.log(a.startsWith(`E`)); // true
16 console.log(a.startsWith(`E`, 2)); // false
17
18 /*
19 endsWith()
20 → تتحقق هل النص ينتهي بقيمة معينة
21 → length تعتمد على
22 → يمكن تحديد طول الفحص
23 */
24 console.log(a.endsWith(`l`)); // true
25 console.log(a.endsWith(`l`, a.length)); // true
```

7-Comparison:

They are coefficients (operators) used to compare two values, and return the result of either true or false.

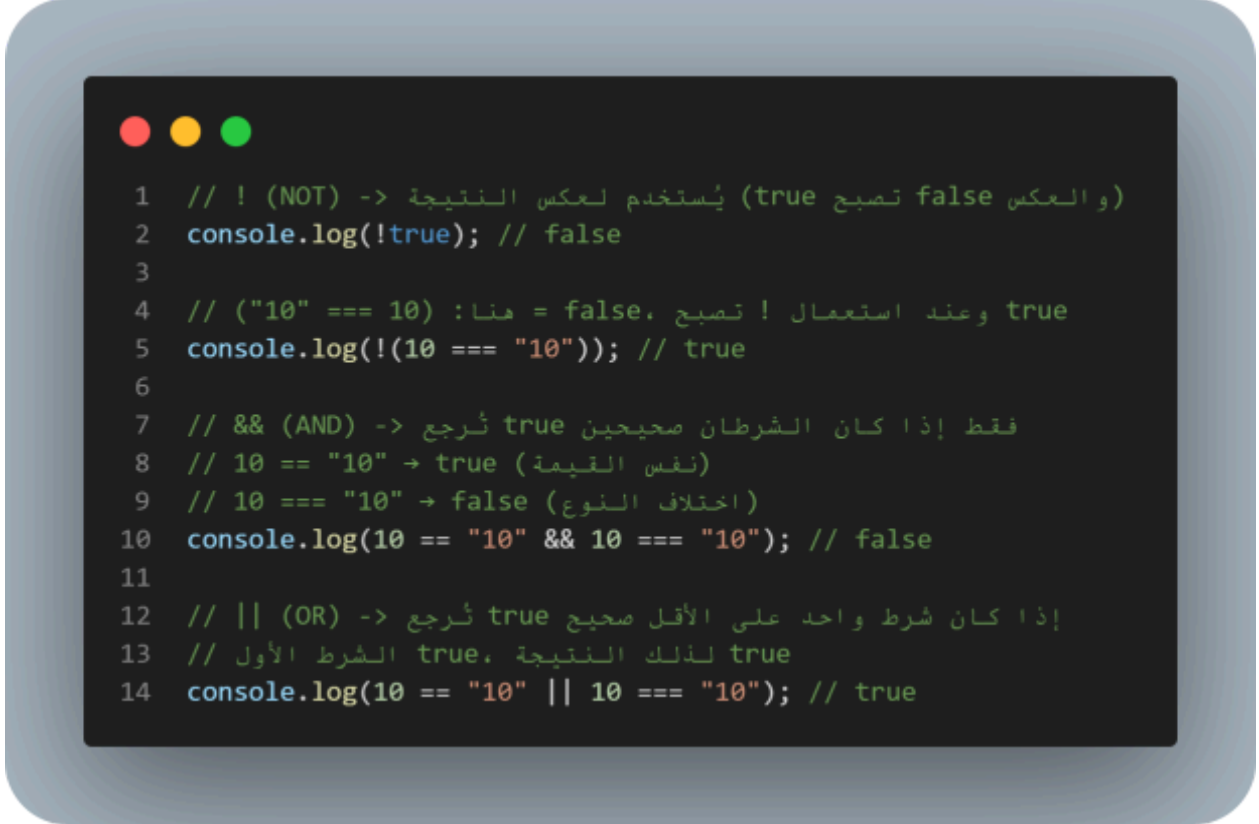
Example:

```
1 // == (يساوي) -> يقارن القيمة فقط دون النظر إلى النوع
2 console.log(10 == "10"); // true
3
4 // != (لا يساوي) -> يتحقق هل القيم غير متساوية (من حيث القيمة فقط)
5 console.log(10 != "10"); // false
6
7 // === (يساوي تماماً) -> يقارن القيمة والنوع معاً
8 console.log(10 === "10"); // false
9
10 // !== (لا يساوي تماماً) -> يتحقق هل القيم أو الأنواع مختلفة
11 console.log(10 !== "10"); // true
12
13 // > (أكبر من)
14 console.log(20 > 10); // true
15
16 // >= (أكبر من أو يساوي)
17 console.log(20 >= 20); // true
18
19 // < (أصغر من)
20 console.log(20 < 40); // true
21
22 // <= (أصغر من أو يساوي)
23 console.log(40 <= 40); // true
24
25 // Example
26 // typeof -> ثرجع نوع البيانات
27 // هنا نقارن نوع سلسلتين نصيتين
28 console.log(typeof "hatim" === typeof "morad"); // true
```

8-Logical Operators:

These are parameters used to concatenate more than one condition together, and return either true or false.

Example:



```
1 // ! (NOT) -> يُستخدم لعكس النتيجة (true تصبح false والعكس)
2 console.log(!true); // false
3
4 // ("10" === 10) : هنا = false، تصبح ! true وعند استعمال ! تصبح
5 console.log(!(10 === "10")); // true
6
7 // && (AND) -> فقط إذا كان الشرطان صحيحين true تُرجع
8 // 10 == "10" → true (نفس القيمة)
9 // 10 === "10" → false (اختلاف النوع)
10 console.log(10 == "10" && 10 === "10"); // false
11
12 // || (OR) -> إذا كان شرط واحد على الأقل صحيح true تُرجع
13 // true لذلك النتيجة، true الشرط الأول
14 console.log(10 == "10" || 10 === "10"); // true
```

9-Condition:

It is a method of making a decision in a program, where a specific code is executed if a specific condition is met.

If: Used to execute code if the condition is true.

Else: If condition is true → executes if

If error → executes else

Else if: Used when we have more than one condition.

```
1
2 let price = 100; // السعر الأصلي
3 let discount = false; // هل يوجد تخفيض
4 let Amountdiscount = 30; // قيمة التخفيض
5 let country = `KSA`; // الدولة
6
7 // إذا كان هناك تخفيض (true)
8 if (discount === true) {
9     price -= Amountdiscount; // طرح قيمة التخفيض من السعر
10
11     // إذا لم يوجد تخفيض، نتحقق من الدولة
12 } else if (country === `Morocco`) {
13     price -= 50; // 50 كانت المغرب نطرح
14
15 } else if (country === `syria`) {
16     price -= 90; // 90 كانت سوريا نطرح
17
18     // إذا لم يتحقق أي شرط
19 } else {
20     price -= 95; // 95 كخيار افتراضي نطرح
21 }
22
23 // عرض السعر النهائي
24 console.log(price); // النتيجة: 5
```

9-1-Nested Condition:

It is placing an if condition inside another if condition, a condition inside a condition, Used when we need to check more than one level of conditions.

```
1 let price1 = 100; // السعر الأصلي
2 let discount1 = false; // هل يوجد تخفيض عام
3 let Amountdiscount1 = 30; // قيمة التخفيض
4 let country1 = `Morocco`; // الدولة
5 let student = true; // هل الشخص طالب
6
7 // إذا كان هناك تخفيض عام
8 if (discount1 === true) {
9     price1 -= Amountdiscount1; // نطرح قيمة التخفيض
10
11     // إذا لم يوجد تخفيض عام، نتحقق من الدولة
12 } else if (country1 === `Morocco`) {
13
14     // شرط متداخل (Nested If)
15     // إذا كان الشخص طالب
16     if (student === true) {
17         price1 -= Amountdiscount1 + 40; // تخفيض كبير (30 + 40)
18
19     } else {
20         // إذا لم يكن طالب
21         price1 -= Amountdiscount1 + 10; // تخفيض أقل (30 + 10)
22     }
23
24     // إذا لم تتحقق أي حالة
25 } else {
26     price1 -= 95; // تخفيض افتراضي
27 }
28
29 // عرض النتيجة النهائية
30 console.log(price1); // النتيجة: 30
```

9-2-Conditional Ternary Operator:

It is an abbreviation for writing an if / else condition in one line.

Example:

```
1 // Conditional Ternary Operator
2 // الشكل: condition ? value_if_true : value_if_false
3
4 let theName = `Ahmed`; // الاسم
5 let theGender = `Male`; // الجنس
6 let theAge = 30; // العمر
7
8 // الجنس: إذا كان Male نطبع "Mr" وإلا "Mrs"
9 theGender == `Male` ? console.log("Mr") : console.log("Mrs");
10
11 // الطريقة 1: تخزين النتيجة في متغير ثم عرضها
12 var Result = theGender == `Male` ? "Mr" : "Mrs";
13 document.writeln(Result);
14
15 // الطريقة 2: نفس الفكرة (تكرار نفس الحل)
16 var Result = theGender == `Male` ? "Mr" : "Mrs";
17 document.writeln(Result);
18
19 // الطريقة 3: طباعة مباشرة بدون متغير
20 console.log(theGender == `Male` ? "Mr" : "Mrs");
21
22 // الطريقة 4: استعمال التيرنري داخل النص (Template String)
23 console.log(`hello ${theGender == `Male` ? "Mr" : "Mrs"} ${theName}`);
24
25 // الطريقة 5: (Nested Ternary) شروط متعددة حسب العمر
26 theAge < 20
27   ? console.log("Less than 20")
28   : theAge > 20 && theAge < 60
29     ? console.log("20 to 60")
30     : theAge > 60
31       ? console.log("Larger than 60")
32       : console.log("Unknown");
```

9-3- CNullish Coalescing Operator And Logical Or:

CNullish Coalescing: It is a parameter used to return the first value if it is not null or undefined only, and ignores the rest of the values such as 0, "" and false.

Logical Or: It is a parameter used to return the first "true" value (truthy) among the values, and considers all values such as false, 0, "", null, undefined to be invalid values.

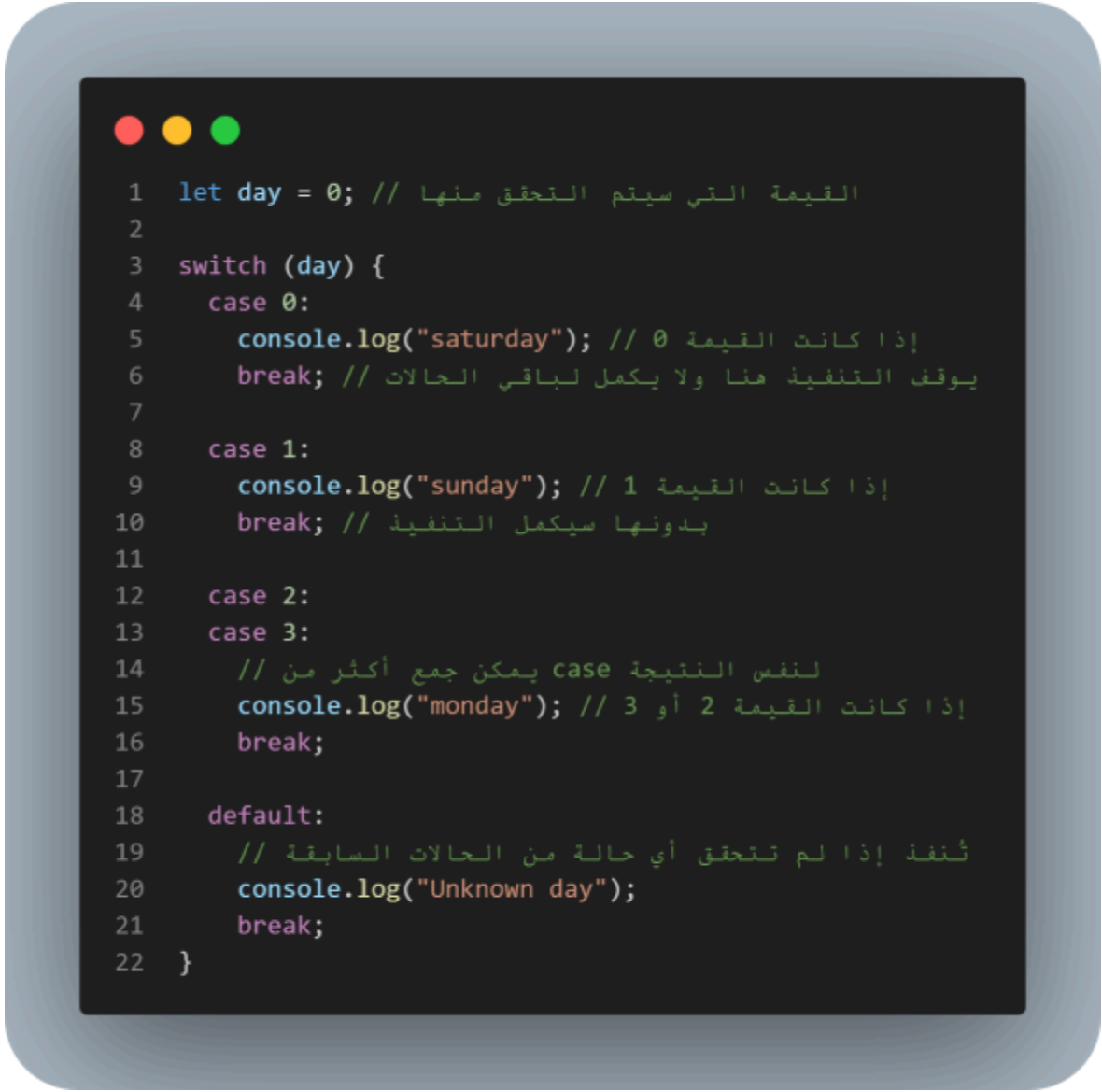
Example:

```
1  let money = 0;
2
3  // || (Logical OR)
4  // كيرجع أول قيمة truthy
5  // 0, "", false, null, undefined = false
6  console.log(`The price is ${money || 1000}`);
7  // النتيجة: 1000 لأن 0 تعتبر false
8
9  // ?? (Nullish Coalescing)
10 // null و undefined كيتجاهل غير
11 // كقيم عادية false و يقبل 0 و "" و
12 console.log(`The price is ${money ?? 1000}`);
13 // money = 0 النتيجة: 0 لأن null و undefined ليست
```

9-4-Switch Statement:

It is a method of choosing to execute a code from among several states (cases) according to a certain value, and it is an alternative to if / else if.

Example:



```
1  let day = 0; // القيمة التي سيتم التحقق منها
2
3  switch (day) {
4      case 0:
5          console.log("saturday"); // إذا كانت القيمة 0
6          break; // يوقف التنفيذ هنا ولا يكمل لباقي الحالات
7
8      case 1:
9          console.log("sunday"); // إذا كانت القيمة 1
10         break; // بدونها سيكمل التنفيذ
11
12     case 2:
13     case 3:
14         // لنفس النتيجة case يمكن جمع أكثر من
15         console.log("monday"); // إذا كانت القيمة 2 أو 3
16         break;
17
18     default:
19         // تُنفذ إذا لم تتحقق أي حالة من الحالات السابقة
20         console.log("Unknown day");
21         break;
22 }
```


10-Loop For:

It is a loop used to repeat the execution of a specific code a specified number of times.

Why use loop for:

- To avoid repeating the same code multiple times.
- To perform the same operation on a set of elements (such as array).
- When we know how many times we want to repeat.
- To make the code shorter and easier to read.

Example:

```
1  /* Loop For
2     تستخدم لتكرار الكود عدد محدد من المرات
3     start index: 0 البداية من
4     condition: 10 التكرار حتى أقل من 10
5  */
6
7  for (let i = 0; i < 10; i++) {
8
9      // (i++) يبدأ من 0 ويزيد بواحد في كل دورة i
10     // i < 10 الحلقة ستستمر ما دام
11
12     console.log(i); // طباعة قيمة i في كل مرة
13 }
```

10-1-Loopin On Sequences:

It is the use of loops (such as for) to pass over ordered elements such as array or string and execute code on each element.

Example:

```
1 let Myfriend = [1, 2, "Hatim", "Morad", "Bouchra", "Rachid", "Israe"];
2
3 // بدون loop (الوصول اليدوي)
4 // كندبروه بوحده index كل
5 console.log(Myfriend[0]);
6 console.log(Myfriend[1]);
7 console.log(Myfriend[2]);
8 console.log(Myfriend[3]);
9 console.log(Myfriend[4]);
10
11 /* =====
12     Static Loop (حلقة ثابتة)
13     ===== */
14
15 // كتحدد عدد التكرارات يدوياً (5 مرات فقط)
16 for (let i = 0; i < 5; i++) {
17     console.log(`${i} - ${Myfriend[i]}`);
18 }
19
20 /* =====
21     Dynamic Loop (حلقة ديناميكية)
22     ===== */
23
24 // باش تدور على جميع العناصر تلقائياً length كاستعمل
25 for (let i = 0; i < Myfriend.length; i++) {
26     console.log(`${i} - ${Myfriend[i]}`);
27 }
```

10-2-Nested loop for:

It is a for loop inside another for loop, and is used when we need to pass data within data (such as an array within an array).

Example:

```
1  Nested For Loop (حلقة متداخلة)
2  - i = الصفوف
3  - j = الأعمدة
4  */
5
6  for (let i = 1; i <= 3; i++) {
7
8      console.log("Row " + i);
9      // غادي يطبع:
10     // Row 1
11     // Row 2
12     // Row 3
13
14     for (let j = 1; j <= 3; j++) {
15
16         console.log("  Column " + j);
17         // غادي يطبع لكل Row:
18         // Column 1
19         // Column 2
20         // Column 3
21     }
22 }
```

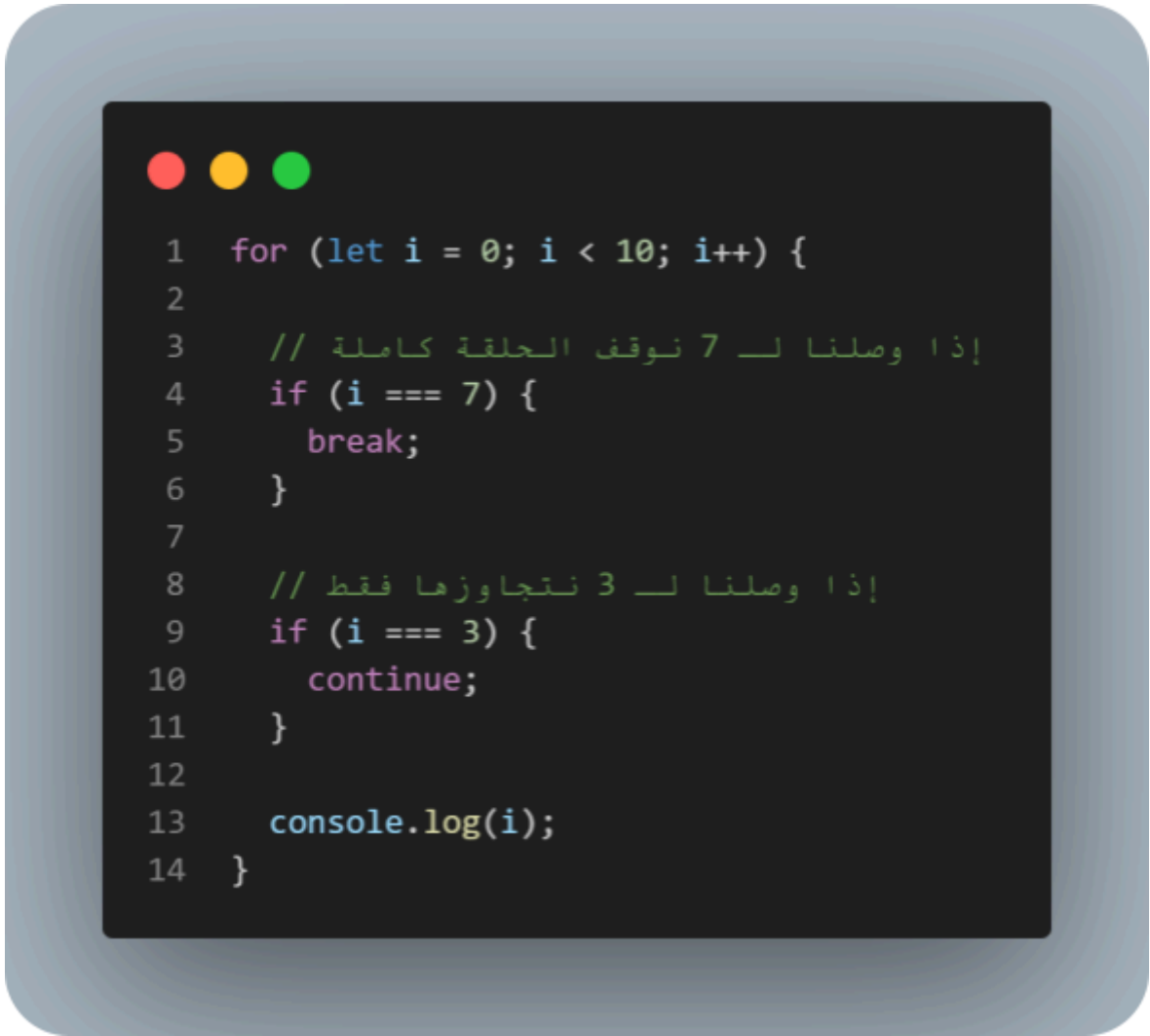
10-3-Loop Control - Break, Continue, Label:

These are tools that we use to control loops (loops) during implementation.

Break: As a final stop to the entire episode.

Continue: It makes the episode skip one cycle and then continue again.

Example:



```
1  for (let i = 0; i < 10; i++) {  
2  
3      // إذا وصلنا لـ 7 نوقف الحلقة كاملة  
4      if (i === 7) {  
5          break;  
6      }  
7  
8      // إذا وصلنا لـ 3 نتجاوزها فقط  
9      if (i === 3) {  
10         continue;  
11     }  
12  
13     console.log(i);  
14 }
```

11-Loop While:

It is a loop used to repeat the execution of a specific code as long as the condition is true (true), and it is distinguished from for in that it is used when we do not know the number of repetitions in advance.

Example:

```
1  /* Loop While */
2
3  let Product11 = ["Mouse", "keyboard", "monitor", "CPU", "GPU"];
4  // مصفوفة فيها 5 عناصر
5  let index = 0; // البداية من أول عنصر
6  while (index < 10) {
7      // أقل من 10 الحلقة ستستمر ما دام
8
9      console.log(Product11[index]);
10     // طباعة العنصر حسب index
11     // ⚠️ (4 → 0) عناصر 5 غير لكن المصفوفة فيها غير
12     // undefined من بعد غادي يطبع
13
14     index += 1; // في كل دورة index زيادة
15
16     if (index === 3) {
17         break; // إلى 3 index يوقف الحلقة عندما يصل
18     }
19 }
```

11-1-Loop Do While:

It is a loop that executes the code at least once and then checks the condition, unlike While which checks the condition first and may never execute the code if the condition is false.

Example:

```
1  /* Loop Do While */
2
3  let i = 0; // البداية من 0
4
5  do {
6      console.log(i);
7      // طباعة قيمة i في كل دورة
8
9      i += 1;
10     // زيادة i مرة في كل واحد
11
12 } while (i < 10);
13 // أقل من 10 i يستمر التكرار ما دام
14
15 // بعد انتهاء الحلقة
16 console.log(i);
17 // بعد الخروج من الحلقة i طباعة القيمة النهائية لـ
```

12-Function:

It is a set of commands (code) that are grouped in one place to be reused when needed.

Why use Function?:

- To avoid duplicate code
- To organize the program
- To reuse the same code more than once

12-1-Built in function VS user defined function:

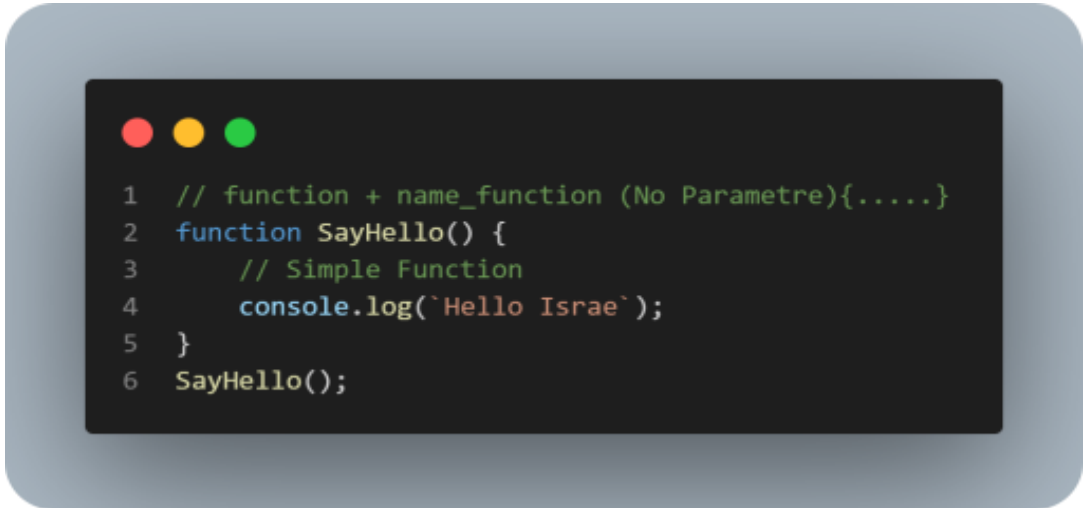
Built in function: They are ready-made functions that originally exist in JavaScript and you use them directly without defining them.

user defined function: They are functions that the programmer creates himself to perform a specific task.

12-2-Function Simple and Parameter:

Function Simple: It is a function without parameters that executes a specific code only when called.

Example:



```
1 // function + name_function (No Parametre){.....}
2 function SayHello() {
3     // Simple Function
4     console.log(`Hello Israe`);
5 }
6 SayHello();
```

Function Parameter:

It is a function that takes inputs (values) from outside and uses them inside the function to perform a specific operation.

Example:



```
1 function Sayhelo(name) {  
2     // name = parameter (مدخل داخل الدالة)  
3     console.log(`Hello ${name}`);  
4     // كيطبع رسالة فيها الاسم اللي دخلناه  
5 }  
6  
7 // استدعاء الدالة مع تمرير قيم (arguments)  
8 Sayhelo(`Israe`); // argument = Israe  
9 Sayhelo(`Bouchra`); // argument = Bouchra  
10 Sayhelo(`Hanae`); // argument = Hanae
```


Function Default Parameter:

It is a default value within a function that is used if no value is passed (argument).

Example:



```
1  function greet(name = "Guest") {  
2      // name = parameter مع قيمة افتراضية "Guest"  
3      // إذا ما تمّش تمرير قيمة، يستعمل "Guest"  
4      console.log(`Hello ${name}`);  
5  }  
6  
7  // هنا مررنا قيمة "Hatim"  
8  greet("Hatim");  
9  // النتيجة: Hello Hatim  
10  
11 // هنا لم نمرر أي قيمة  
12 greet();  
13 // (استعمل القيمة الافتراضية) النتيجة: Hello Guest
```

Function Rest Parameters:

It is a method within a function that allows you to add an unlimited number of values into a single Array.

Example:



```
1 function sum(...numbers) {  
2     // ...numbers = Rest Parameter  
3     // واحدة Array كيجمع جميع القيم الممررة في  
4     console.log(numbers);  
5 }  
6  
7 // تمرير عدة قيم  
8 sum(1, 2, 3, 4);  
9  
10 // النتيجة: [4, 3, 2, 1]
```

12-3-Anonymous Function :

It is a function without a name, that is stored in a variable or used directly to perform a specific task.

Why use it?

- When we do not need a name for the function.
- When the function is stored in a variable.
- callbacks

Example:



```
1  let calc = function (num1, num2) {  
2      // Anonymous Function (دالة بدون اسم)  
3      // num1 و num2 = parameters  
4  
5      return num1 + num2;  
6      // ترجع مجموع الرقمين  
7  };  
8  
9  // استدعاء الدالة مع تمرير قيم (arguments)  
10 console.log(calc(20, 30));  
11  
12 // النتيجة: 50
```

12-4-Arrow Function :

It is an abbreviated way to write functions (functions) using the arrow => instead of the word function.

Example:

```
1  /* صيغة Arrow Function:
2     إذا كان لديك تعبير واحد (سطر واحد) يمكن اختصار الكود
3  */
4
5  let print = num => num;
6  // num = معامل (parameter)
7  // () إذا كان لديك معامل واحد يمكن حذف الأقواس
8  // return ويتم إرجاع القيمة مباشرة بدون استخدام
9
10 console.log(print(100));
11 // النتيجة: 100
12
13
14 let Addnum = (num, num2) => num + num2;
15 // هنا لدينا أكثر من معامل
16 // لذلك يجب استعمال الأقواس
17 // (return ضمني) ويتم إرجاع نتيجة الجمع مباشرة
18
19 console.log(Addnum(100, 50));
20 // النتيجة: 150
```

13- Higher order Function:

is a function that takes another function as an argument or returns a function.

13-1- Map:

is a method used to modify array elements and create a new array.

```
1 // Higher Order Functions - Map(Anonymous Function)
2 // map كندور على كل عنصر في array
3 // element = العنصر الحالي
4 // index = رقم العنصر
5 // array = كاملة array
6 let AddSelf = MyNums.map(function (element, index, array) {
7     // كنضرب كل عنصر في 2
8     return element * 2
9 }, 10)
10 console.log(AddSelf)
11
12 // Higher Order Functions - Map(Arrow Function)
13 // Arrow Function نفس الفكرة ولكن باستعمال
14 let AddSelf1 = MyNums.map((element) => element * 2)
15 console.log(AddSelf1)
16
17
18 // Higher Order Functions - Map(Regular Function)
19 // Function عادية
20 function addition(ele) {
21     // كنرجع العنصر مضروب في 2
22     return ele * 2
23 }
24
25 // map function كتنفذ على كل عنصر
26 let add = MyNums.map(addition)
27 console.log(add)
```

13-2-ForEach:

is a method used to loop through array elements and execute a function on each one without returning a new array.

Example:

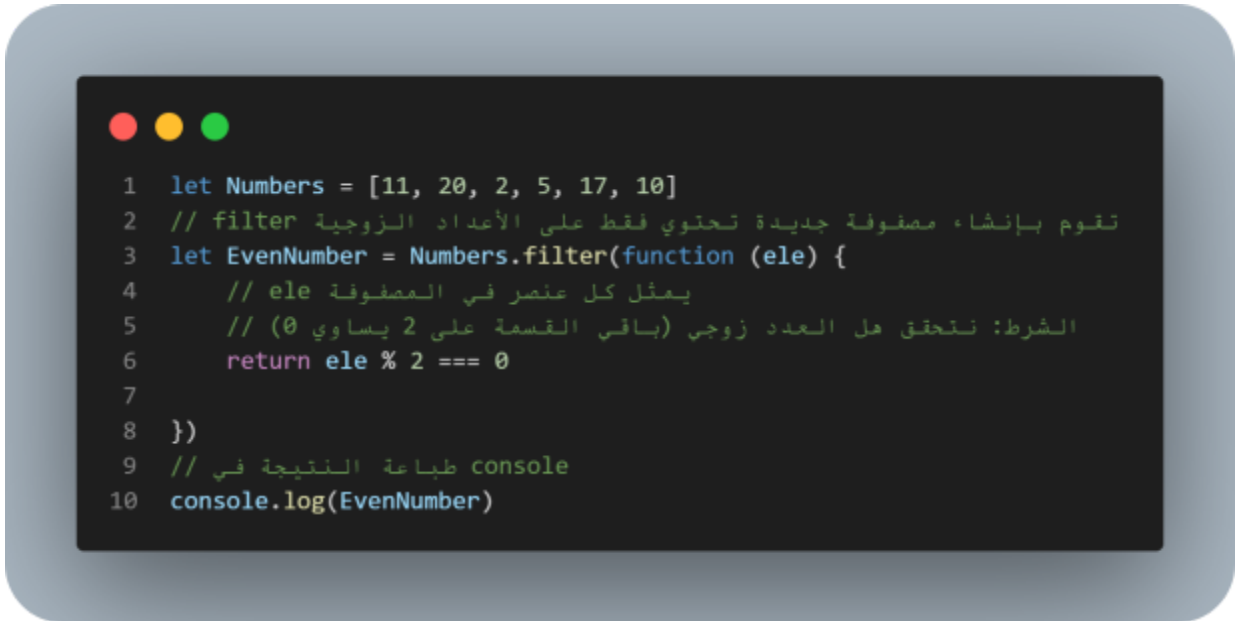


```
1 let nums = [5, 10, 15]
2 // forEach كـتدور على كل عنصر فـ array
3 nums.forEach(function (element, index, array) {
4     // element = القيمة ديال العنصر الحالي
5     console.log("value:", element)
6     // index = رقم العنصر
7     console.log("index:", index)
8     // array = كاملة array
9     console.log("array:", array)
10 })
```

13-3- Filter:

is a method used to select elements from an array based on a condition and returns a new array with only the matching elements.

Example:

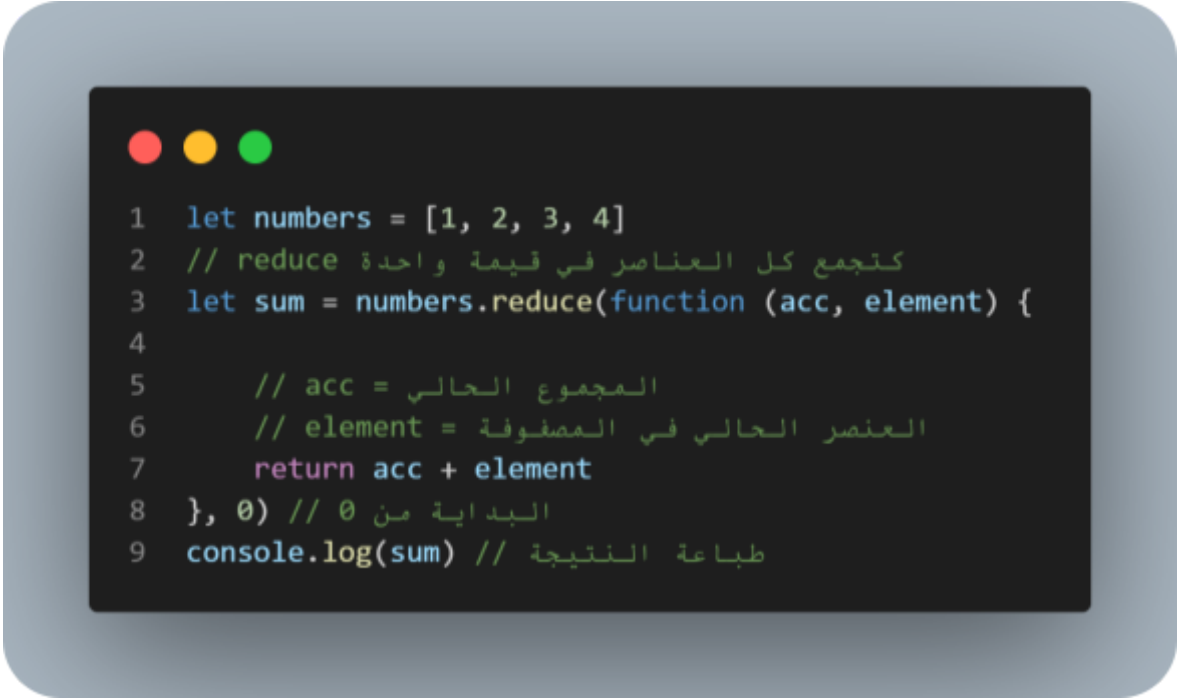


```
1 let Numbers = [11, 20, 2, 5, 17, 10]
2 // filter مصفوفة جديدة تحتوي فقط على الأعداد الزوجية
3 let EvenNumber = Numbers.filter(function (ele) {
4     // ele يمثل كل عنصر في المصفوفة
5     // الشرط: نتحقق هل العدد زوجي (باقي القسمة على 2 يساوي 0)
6     return ele % 2 === 0
7
8 })
9 // طباعة النتيجة في console
10 console.log(EvenNumber)
```

13-4-Reduce:

is a method used to reduce an array to a single value (like sum, product, etc.)

Example:



```
1 let numbers = [1, 2, 3, 4]
2 // reduce كتجمع كل العناصر في قيمة واحدة
3 let sum = numbers.reduce(function (acc, element) {
4
5     // acc = المجموع الحالي
6     // element = العنصر الحالي في المصفوفة
7     return acc + element
8 }, 0) // 0 البداية من
9 console.log(sum) // طباعة النتيجة
```


>Data Structure:

1- Array:

It is a type of data used to store a set of values in a single variable.

Example:

```
1  /* Intro Array */
2
3  // Simple Array
4  let Names = [`Hatim`, `Mohamed`, `Morad`];
5  // مصفوفة تحتوي على 3 أسماء
6
7  console.log(`Hello ${Names[0]}`);
8  // الوصول لأول عنصر في المصفوفة (Hatim)
9  // النتيجة: Hello Hatim
10
11
12 console.log(`${Names[1][1]}`.toUpperCase());
13 // Names[1] = "Mohamed"
14 // Names[1][1] = الحرف الثاني → "o"
15 // toUpperCase() = تحويله إلى حرف كبير → "O"
16 // النتيجة: O
17
18
19 Names[0] = `gamal`;
20 // Hatim تغيير أول عنصر في المصفوفة من
21
22 console.log(Names);
23 // طباعة المصفوفة بعد التغيير
24 // النتيجة: ["gamal", "Mohamed", "Morad"]
```

1-1-Nested Array:

It is an array that contains another array inside it.

Example:

```
1 // Nested Array
2
3 let friends = ['Hatim', 'Mohamed', 'Morad', ['Israe', 'Hanae', 'Bouchra']];
4 // مصفوفة تحتوي على 3 عناصر + مصفوفة داخلية
5
6 console.log(`Hello ${friends[3][0]}`);
7 // friends[3] = ["Israe", "Hanae", "Bouchra"]
8 // friends[3][0] = "Israe"
9 // النتيجة: Hello Israe
10
11
12 console.log(`${friends[3][0][0]}.toUpperCase()`);
13 // friends[3][0] = "Israe"
14 // friends[3][0][0] = أول حرف → "I"
15 // toUpperCase() = تحويله إلى حرف كبير
16 // النتيجة: I
```

Add info:

`console.log(Array.isArray(Names));` // for check this is array

1-2-Index and Length With Array:

Index: Index is the location number of the element within the Array, always starting from 0 and not 1.

Length: It is a property that gives you the number of elements inside an array.

```
1
2  /* Using Length With Array */
3
4  // Index [3, 2, 1, 0] → يبدأ من 0
5  // Length [4, 3, 2, 1] → هو عدد العناصر
6
7  let fruited = ['banan', 'apple', 'orange', 'dragon'];
8  // مصفوفة فيها 4 عناصر
9
10 console.log(fruited.length);
11 // النتيجة: 4 (عدد العناصر)
12
13 // بعيد index إضافة عنصر في
14 fruited[7] = 'manga';
15 // هنا أضفنا عنصر في index = 7
16 // index 4 و 5 و 6 سيكونوا empty (undefined)
17
18 console.log(fruited);
19 // النتيجة:
20 // ["banan", "apple", "orange", "dragon", empty × 3, "manga"]
21
22 console.log(fruited.length);
23 // النتيجة: 8
24 // length = 8 → هو index 7 لأن آخر
```

1-3- Methode Array:

They are ready-made functions used to deal with matrices such as addition, deletion, arrangement, and search.

1-3-1- Add And Remove From Array:

Unshift: Used to add an element at the beginning of an array

Push: Used to add an element at the end of an array.

Shift: Used to delete the first element of an array.

Pop: Used to delete the last element of an array.

```
1  /* Add And Remove From Array */
2  let cars = [`lambo`, `ferarai`, "toyota", `pegeute`];
3  console.log(cars);
4
5  cars.unshift(`hyundai`); // add item in start array
6  console.log(cars);
7
8  cars.push(`KIA`); // add item in end array
9  console.log(cars);
10
11 cars.shift(); // Remove item in start array
12 console.log(cars);
13
14 cars.pop(); // Remove item in end array
15 console.log(cars);
16
17 let last = cars.pop();
18 console.log(`this cars ${last}`);
```

1-3-2-Searching Array:

These are methods (methods) used to search for an element within an array and find out its location or presence.

```
1 let laptop = ['Asus', 'gigabayte', 'Hp', 'Dell'];
2 console.log(laptop);
3
4 // indexOf:
5 // تبحث عن عنصر داخل المصفوفة من اليسار إلى اليمين
6 // إذا وجدته، وإذا لم يوجد ترجع -1 (الموقع) index ترجع
7 // (index) يمكن تحديد من أين يبدأ البحث
8 console.log(laptop.indexOf('Dell', 1));
9 // index يبدأ من 1
10 // النتيجة: 3
11
12 // lastIndexOf:
13 // تبحث عن العنصر من اليمين إلى اليسار
14 // للعنصر (index) ترجع آخر موقع
15 // يمكن أيضاً تحديد نقطة البداية
16 console.log(laptop.lastIndexOf('Dell'));
17 // النتيجة: 3
18
19 console.log(laptop.lastIndexOf('gigabayte', -2));
20 // يعني يبدأ من قبل آخر عنصر بمقدارين -2
21 // تبحث من اليمين إلى اليسار
22
23 // includes:
24 // تتحقق هل العنصر موجود أم لا
25 // إذا لم يوجد false وإذا وجد، و true ترجع
26 // يمكن تحديد من أين يبدأ البحث
27 console.log(laptop.includes('Dell')); // true
28 console.log(laptop.includes('Dell', 1));
29 // index يبدأ البحث من 1
30
31 // إذا لم يوجد العنصر:
32 // يرجعان 1- lastIndexOf و indexOf
33 // includes ترجع false
```

1-3-3-Sorting Array:

It is the process of arranging the elements of a matrix (ascending or descending).

Example:

```
1  /* Sorting Arrays */
2
3  let random = [10, 'ahmed', 'fatih', '100', 90, 45, '10000', -2929];
4  console.log(random);
5  // طباعة المصفوفة كما هي
6
7  // sort():
8  // تقوم بترتيب عناصر المصفوفة
9  // ⚠ ليس كأرقام (string) لكن الترتيب يكون كنصوص (ASCII)
10 // يعني تعتمد على الترتيب الأبجدي
11 console.log(random.sort());
12 // مثال: "100" تأتي قبل "2" لأن المقارنة تكون حرفية
13
14 // reverse():
15 // تقوم بعكس ترتيب العناصر في المصفوفة
16 // للحصول على ترتيب تنازلي sort غالباً تُستعمل بعد
17 console.log(random.reverse());
```

1-3-4-Array Operations (Slice, Splice, Concat, Join) :

It is a set of functions in JavaScript used to manipulate arrays, such as copying part of them, deleting or adding elements, merging several arrays, or converting them to text

Example:



```
1  /* Slicing Array */
2  let teachers = ['Imad', 'Hatim', 'Zakaria', 'abo', 'youssef', 'l7aj'];
3  console.log(teachers);
4
5  // slice:
6  // نستخدم لنسخ جزء من المصفوفة بدون تغيير المصفوفة الأصلية
7  // Positive Number
8  console.log(teachers.slice(1));
9  // إلى النهاية index 1 يبدأ من
10
11 console.log(teachers.slice(1, 3));
12 // يبدأ من 1 ويتوقف قبل 3
13 // و 2 فقط index 1 يعني يأخذ
14
15 // Negative Number
16 console.log(teachers.slice(-3));
17 // يأخذ آخر 3 عناصر
18
19 console.log(teachers.slice(1, -3));
20 // ويتوقف قبل آخر 3 عناصر index 1 يبدأ من
21
22 // splice:
23 // نستخدم لحذف أو إضافة عناصر داخل المصفوفة (تُغيّر الأصل)
24
25 console.log(teachers.splice(0, 4, 'issam', 'fatih'));
26 // index 0 يبدأ من
27 // يحذف 4 عناصر
28 // "issam" و "fatih" ثم يضيف
29 // ترجع العناصر التي تم حذفها
30
31
32 /* Joining and concat Arrays*/
33
34 // concat:
35 // نستخدم لدمج عدة مصفوفات معاً في مصفوفة جديدة
36 // لا تُغيّر المصفوفة الأصلية
37 let myfriend = ['Zoubir', 'Hamza', 'Annas', 'Adam', 'fatih', 'abo ri7'];
38 let myNewFriend = ["Saad", 'Haytam'];
39 let mySchoolFriend = ["reda", 'Alae'];
40
41 let AllMyfriend = myfriend.concat(
42   myNewFriend,
43   mySchoolFriend,
44   'Israe',
45   ['Bouchra', 'Hanae']
46 );
47 console.log(AllMyfriend);
48
49 // join:
50 // (string) نستخدم لتحويل المصفوفة إلى نص
51 // مع وضع فاصل بين العناصر
52 console.log(AllMyfriend.join(';'));
53 // هنا الفاصل هو ;
54
```

2- Object:

is a data structure that contains properties (attributes) and functions (methods) to represent a real or logical entity.

Example:

```
1 let user = {
2     // Properties OR Attributes (خصائص الكائن)
3     // تمثيل معلومات عن المستخدم
4     theName: "Hatim", // اسم المستخدم
5     theAge: 18,        // عمر المستخدم
6     // Method (دالة داخل الكائن)
7     // تمثيل سلوك أو فعل يقوم به المستخدم
8     SayHello: function () {
9         return "Hello" // ترجع جملة ترحيب
10    },
11 };
12 // Accessing Object (الوصول إلى خصائص ودوال الكائن)
13 // طباعة اسم المستخدم
14 console.log(user.theName)
15 // طباعة عمر المستخدم
16 console.log(user.theAge)
17 // استدعاء الدالة داخل الكائن وطباعة نتائجها
18 console.log(user.SayHello())
```


2-1- Dot Notation VS Bracket Notation :

Dot Notation: is a way to access object properties using a dot when the property name is simple.

Bracket Notation : is a way to access object properties using square brackets, used for complex keys, String or keys with spaces.

```
1  let user = {
2    name: "Ali",           // خاصية عادية
3    age: 20,               // خاصية عادية
4    "country of": "Egypt", // خاصية فيها مسافة
5    100: "number key"      // مفتاح رقمي
6  }
7  /* -----
8     Dot Notation (.)
9     -----
10    كنستعملها فقط مع المفاتيح البسيطة
11    التي ما فيهاش مسافات ولا رموز
12  */
13  console.log(user.name) // كيطبع: Ali
14  console.log(user.age)  // كيطبع: 20
15  /* -----
16     Bracket Notation []
17     -----
18    كنستعملوها مع:
19    - مفاتيح فيها مسافات
20    - رموز
21    - أرقام
22  */
23  console.log(user["country of"]) // كيطبع: Egypt
24  console.log(user[100])           // كيطبع: number value
```

2-2- Nested Object:

A nested object is an object inside another object as a property value.

```
1 let Availble = true;
2 let Batata = {
3   // اسم الشخص
4   name: `hatim`,
5   // عمر الشخص
6   Age: 18,
7   // لائحة المهارات (Array)
8   Skills: ["HTML", "CSS", "JS"],
9   // كتمثل التوفر object خاصة داخل
10  Availble: false,
11  // فيه العناوين (Nested Object) كائن داخل كائن
12  adreses: {
13    KSA: "Riyadh",
14    Moroco: {
15      one: "Tetouan",
16      two: "Tangier",
17    },
18  },
19 }
20 // طباعة اسم الشخص
21 console.log(Batata.name)
22 // طباعة المهارات مفصولة بـ "\"
23 console.log(Batata.Skills.join("\\"))
24 // طباعة أول مهارة (index 2)
25 console.log(Batata.Skills[2])
26 // الوصول لمدينة في السعودية
27 console.log(Batata.adreses.KSA)
28 // الوصول لمدينة داخل المغرب
29 console.log(Batata.adreses.Moroco.one)
30 // Bracket + Dot notation نفس الوصول ولكن بـ
31 console.log(Batata["adreses"].Moroco.one)
32 // كامل Bracket notation الوصول بـ
33 console.log(Batata["adreses"]["Moroco"]["two"])
```

2-3- Methods Object:

used to work with objects, such as getting keys, values, or manipulating them.

Example:

```
1  let user = {
2    name: "Ali",
3    age: 20,
4    city: "Tanger"
5  }
6
7  /* -----
8    1. Object.keys()
9    كترجع غير المفاتيح (keys)
10   -----*/
11  console.log(Object.keys(user))
12  // ["name", "age", "city"]
13
14  /* -----
15    2. Object.values()
16    كترجع غير القيم (values)
17   -----*/
18  console.log(Object.values(user))
19  // ["Ali", 20, "Tanger"]
20
21  /* -----
22    3. Object.entries()
23    مع بعض key + value كترجع
24   -----*/
25  console.log(Object.entries(user))
26  // [["name","Ali"], ["age",20], ["city","Tanger"]]
```



```
1  /* -----
2    4. Object.freeze()
3    (ما يمكنش نبدلوه) object كيقل
4    -----*/
5    Object.freeze(user)
6    user.name = "Ahmed" // ما غاديش يتبدل
7    console.log(user.name)
8    // Ali
9
10 /* -----
11    5. Object.assign()
12    objects كينسخ أو كيجمع
13    -----*/
14    let extra = {
15        country: "Morocco"
16    }
17    let newUser = Object.assign({}, user, extra)
18    console.log(newUser) // { name, age, city, country }
19
20 /* -----
21    6. Object.hasOwnProperty()
22    object موجودة داخل key كتشوف واش
23    -----*/
24    console.log(Object.hasOwnProperty(user, "name")) // true
25    console.log(Object.hasOwnProperty(user, "email")) // false
26
27 /* -----
28    5. Object.seal()
29    ولكن كيسمح بتعديل القيم object كيقل
30    (keys ما يمكنش نضيفو أو نحيدو)
31    -----*/
32    Object.seal(user)
33
34    user.age = 25 // مسموح
35    user.newKey = "test" // ممنوع
```

3-Map And Set: In Feature.....

>Advanced Concepts:

1-DOM (Document Object Model):

is a tree-like representation of a web page that allows JavaScript to access, change, and interact with HTML and CSS elements.

1-1-DOM Selectors:

are methods used to select HTML elements from the DOM using JavaScript.

```
1 // Find Element By ID
2 // id كنفقاو عنصر واحد باستعمال unique خاص يكون
3 let myIdElement = document.getElementById("my-div");
4 console.log(myIdElement);
5
6 // Find Element By Tag Name
7 // tag كنفقاو جميع العناصر حسب (بحال p, div, h1...)
8 let myTagNameElement = document.getElementsByTagName("p");
9 // index لذلك كنفستعمل HTMLCollection كيرجع
10 console.log(myTagNameElement[1]);
11
12 // Find Element By Class Name
13 // class كنفقاو جميع العناصر التي عندهم نفس
14 let myClassNameElement = document.getElementsByClassName("my-span");
15 // index كيرجع مجموعة لذلك كنفستعمل
16 console.log(myClassNameElement[1]);
17
18 // querySelector
19 // (id, class, tag) كيمكنك تختار أي عنصر
20 // ولكن كيرجع غير أول عنصر كنفقاو
21 let myQueryElement = document.querySelector(".Name");
22 console.log(myQueryElement);
23
24 // querySelectorAll
25 // selector كيرجع جميع العناصر التي كنفطابقو
26 let myQueryAllElement = document.querySelectorAll(".my-span");
27 // index لذلك كنفستعمل NodeList كيرجع
28 console.log(myQueryAllElement[1]);
```



```
1 // الوصول لعناصر أخرى من document
2 // عنوان الصفحة
3 console.log(document.title);
4 // body الصفحة كاملة
5 console.log(document.body);
6 // form داخل input أول
7 console.log(document.forms[0].one);
8 // link الصفحة أول
9 console.log(document.links[0]);
```

1-2-DOM Deal With Children's :

accessing and manipulating elements that are inside another parent element in the DOM.

Example:

```
1 let Parent = document.getElementsByClassName("Five")[0];
2
3 // children
4 // كيرجع غير العناصر (Element nodes فقط)
5 // فقط داخل HTML tags يعني Parent
6 console.log(Parent.children);
7
8 // أول طفل (Element فقط)
9 console.log(Parent.children[0]);
10
11 // childNodes
12 // كيرجع جميع الأنواع:
13 // - Elements
14 // - Text nodes (spaces, line breaks)
15 console.log(Parent.childNodes);
16
17 // childNodes عنصر رقم 3 داخل
18 console.log(Parent.childNodes[3]);
```




```
1 // firstChild
2 // text أو element ممكن يكون (node أول)
3 console.log(Parent.firstChild);
4
5 // lastChild
6 // text أو element ممكن يكون (node آخر)
7 console.log(Parent.lastChild);
8
9 // firstElementChild
10 // text بدون (فقط HTML أول عنصر)
11 console.log(Parent.firstElementChild);
12
13 // lastElementChild
14 // text بدون (فقط HTML آخر عنصر)
15 console.log(Parent.lastElementChild);
```

1-3-DOM Traversing:

It is the navigation between DOM elements such as father (Parent), and siblings (Siblings) to access and control the elements.

```
1  let span = document.querySelector(".two1")
2
3  // nextSibling
4  // موجودة بعد العنصر مباشرة node ترجع أي
5  // يمكن أن تكون:
6  // - HTML عنصر
7  // - نص
8  // - comment
9  console.log(span.nextSibling)
10
11 // nextElementSibling
12 // الموجود بعده مباشرة فقط HTML ترجع العنصر
13 // span أو p أو div مثل
14 console.log(span.nextElementSibling)
15
16 // previousSibling
17 // موجودة قبل العنصر node ترجع أي
18 // يمكن أن تكون:
19 // - نص
20 // - comment
21 // - HTML عنصر
22 console.log(span.previousSibling)
23
24 // parentElement
25 // span ترجع العنصر الأب الذي يحتوي على
26 console.log(span.parentElement)
```

1-4-Get & Set Elements Content And Attributes:

methods and properties used to get or set element content and attributes in the DOM.

```
1 let myElement = document.querySelector(".js")
2
3 // textContent
4 // الحصول على النص داخل العنصر
5 console.log(myElement.textContent)
6
7 // تغيير النص داخل العنصر
8 myElement.textContent = "Hello JavaScript"
9
10 // innerHTML
11 // داخل العنصر HTML الحصول على
12 console.log(myElement.innerHTML)
13
14 // داخل العنصر HTML تغيير
15 myElement.innerHTML = "<span>Hello</span>"
16
17 // getAttribute()
18 // الحصول على قيمة attribute
19 console.log(myElement.getAttribute("class"))
20
21 // setAttribute()
22 // تغيير أو إضافة attribute
23 myElement.setAttribute("title", "New Title")
```



```
1 let myImage = document.querySelector(".image");
2
3 // تغيير خصائص الصورة
4 myImage.src = "https://picsum.photos/200";
5 myImage.alt = "Nature Image";
6 myImage.title = "Beautiful Image";
7
8 // hasAttribute()
9 // التحقق هل الصورة تحتوي على alt
10 console.log(myImage.hasAttribute("alt")); // true
11
12 // removeAttribute()
13 // حذف title من الصورة
14 myImage.removeAttribute("title");
15
16 // مازال موجود title التحقق هل
17 console.log(myImage.hasAttribute("title")); // false
18
19 /* =====
20    التعامل مع الرابط
21    ===== */
22 // اختيار الرابط
23 let myLink = document.querySelector(".link");
24
25 // التحقق هل الرابط يحتوي على href
26 console.log(myLink.hasAttribute("href")); // true
27
28 // حذف title
29 myLink.removeAttribute("title");
30
31 // التحقق من حذف title
32 console.log(myLink.hasAttribute("title")); // false
```

1-5-Class List:

It is a property used to handle classes of HTML elements such as add, delete, check, and switch.

```
1  let element = document.getElementById("my-div");
2
3  /* -----
4     contains()
5     -----
6     موجودة أم لا class التحقق هل
7  */
8  console.log(element.classList.contains("two"));
9
10 /* -----
11    item(index)
12    -----
13    index حسب class ترجع اسم
14  */
15 console.log(element.classList.item(3));
16
17 /* -----
18    add()
19    -----
20    أو أكثر class إضافة
21  */
22 console.log(
23     element.classList.add("hatim", "mohamed")
24 );
```



```
1  /* -----
2      remove()
3      -----
4      حذف class
5  */
6  console.log(
7      element.classList.remove("two")
8  );
9
10 /* -----
11     toggle()
12     -----
13     موجودة <= يتم حذفها class إذا كانت
14     إذا لم تكن موجودة <= يتم إضافتها
15 */
16 console.log(
17     element.classList.toggle("hatim")
18 );
```

1-6-Create And Append Elements:

They are methods used to create new HTML elements and append them to the page using JavaScript.

```
1 // جديد من نوع HTML إنشاء عنصر
2 // جديد ولكن مازال ما تبانش في الصفحة tag يعني كنصايبو
3 let element = document.createElement("div");
4
5 // جديدة (خاصية) إنشاء attribute
6 // ولكن مازال ما تعطاش لعنصر title اسمها attribute هنا صايبنا
7 let Myattr = document.createAttribute("title");
8
9 // إنشاء نص (text node)
10 // مباشرة HTML ماشي داخل DOM يعني نص داخل
11 let mytext = document.createTextNode("Family");
12
13 // DOM تعليق داخل إنشاء comment
14 // ما كيبانش في الصفحة ولكن كيبان في DOM
15 let mycomment = document.createComment("this mohamed stupid");
```

How to append in Page?

```
1 // إضافة النص داخل الـ div
2 div.append(text);
3
4 // إضافة الـ div إلى الصفحة
5 document.body.append(div);
```



```
1  let element2 = document.getElementById("my-div2");
2
3  // إنشاء عنصر جديد (p)
4  let createdP = document.createElement("p");
5
6  /* -----
7     before()
8     -----
9     كضيف عنصر أو نص قبل العنصر
10    (before element) ولكن خارج العنصر
11  */
12  element2.before("Hello From JS");
13
14  /* -----
15     after()
16     -----
17     كضيف عنصر أو نص بعد العنصر
18    (after element) ولكن خارج العنصر
19  */
20  element2.after("Hello from HTML");
21
22  // بعد العنصر (p) إضافة عنصر جديد
23  element2.after(createdP);
```




```
1  /* -----
2      append()
3      -----
4      كضيف عنصر أو نص داخل العنصر
5      (inside end) في الأخير
6  */
7  element2.append("hatim");
8
9  /* -----
10     prepend()
11     -----
12     كضيف عنصر أو نص داخل العنصر
13     (inside start) في البداية
14 */
15 element2.prepend("mohamed");
16
17 /* -----
18     remove()
19     -----
20     كيمسح العنصر كامل من الصفحة
21     DOM (inspect) حتى من
22 */
23 element2.remove();
```

1-7-DOM CSS:

change or control CSS styles of HTML elements dynamically.

```
1 let element1 = document.getElementById("my-div1");
2
3 /* =====
4  الطريقة الأولى لتغيير
5  =====
6  نستعمل style.property
7  نكتبها بـ camelCase كل خاصية
8  */
9 element1.style.color = "green"; // تغيير اللون
10 element1.style.fontWeight = "bold"; // جعل الخط غليظ
11
12 /* =====
13  الطريقة الثانية (cssText)
14  =====
15  كامل كسلسلة نصية CSS نكتب
16  السابق CSS يتم استبدال كل
17  */
18 element1.style.cssText = "color: red; opacity: 0.5";
19
20 /* =====
21  removeProperty()
22  =====
23  من العنصر حذف خاصية
24  */
25 element1.style.removeProperty("color");
26
27 /* =====
28  setProperty()
29  =====
30  إضافة أو تعديل خاصية
31  !important مع إمكانية استعمال
32  */
33 element1.style.setProperty("font-size", "100px", "important");
```



```
1  /* =====
2  CSS (styleSheets) التعامل مع ملف
3  =====
4  (style.css) من ملف خارجي CSS نقدر نعدل
5  */
6
7  /* في أول rule حذف خاصية من أول stylesheet */
8  document.styleSheets[0].cssRules[0].style.removeProperty("line-height");
9
10 /* خارجي CSS تعديل خاصية في */
11 document.styleSheets[0].cssRules[0].style.setProperty(
12     "background-color",
13     "red"
14 );
```

1-8-DOM Event and AddEventListener:

Event:

is an action that happens in the browser like clicking a button, typing, scrolling, or loading a page, and JavaScript uses them to respond to user interactions.

Important Event:

1-8-1-Event Mouse:

```
1 // click
2 // كيقوع ملى المستخدم كيدير ضغطة واحدة على العنصر
3 element.addEventListener("click", function () {
4     console.log("clicked")
5 })
6
7 // dblclick
8 // كيقوع ملى المستخدم كيدير جوج ضغطات بسرعة (double click)
9 element.addEventListener("dblclick", function () {
10     console.log("double clicked")
11 })
12
13 // mouseover
14 // كيقوع ملى الفأرة كتدخل فوق العنصر (hover عليه)
15 element.addEventListener("mouseover", function () {
16     console.log("mouse over")
17 })
18
19 // mouseout
20 // كيقوع ملى الفأرة كتخرج من فوق العنصر
21 element.addEventListener("mouseout", function () {
22     console.log("mouse out")
23 })
```

1-8-2-Form Event:

```
1 let input = document.querySelector("input")
2
3 // أثناء الكتابة
4 input.addEventListener("input", function () {
5   console.log(input.value)
6 })
7
8 // عند تغيير القيمة
9 input.addEventListener("change", function () {
10  console.log("changed")
11 })
12
13 // عند إرسال النموذج
14 document.querySelector("form").addEventListener("submit", function (e) {
15   e.preventDefault() // منع إعادة تحميل الصفحة
16   console.log("form submitted")
17 })
```

1-8-3-Window Event:

```
1 // عند تحميل الصفحة
2 window.addEventListener("load", function () {
3   console.log("page loaded")
4 })
5
6 // عند التمرير
7 window.addEventListener("scroll", function () {
8   console.log("scrolling")
9 })
10
11 // عند تغيير حجم الشاشة
12 window.addEventListener("resize", function () {
13   console.log("resized")
14 })
```

2-BOM(Browser Object Model):

is a model that allows JavaScript to interact with the browser itself (window, alerts, history, etc.), not just the HTML page.

2-1-SetTimeout and clearTimeout:

SetTimeout: is a function that runs code after a specified delay.



```
1  /* =====
2      setTimeout()
3      =====
4      بعد مدة زمنية معينة function كتنفذ
5  */
6
7  let timer = setTimeout(function () {
8      console.log("Hello after 3 seconds")
9  }, 3000)
10 // 3000 = 3 seconds
11
12 // لفنكشن parameters تقدر تزيد
13 setTimeout(function (name) {
14     console.log("Hello " + name)
15 }, 2000, "Hatim")
```

clearTimeout: is a function used to cancel a setTimeout() before it runs.

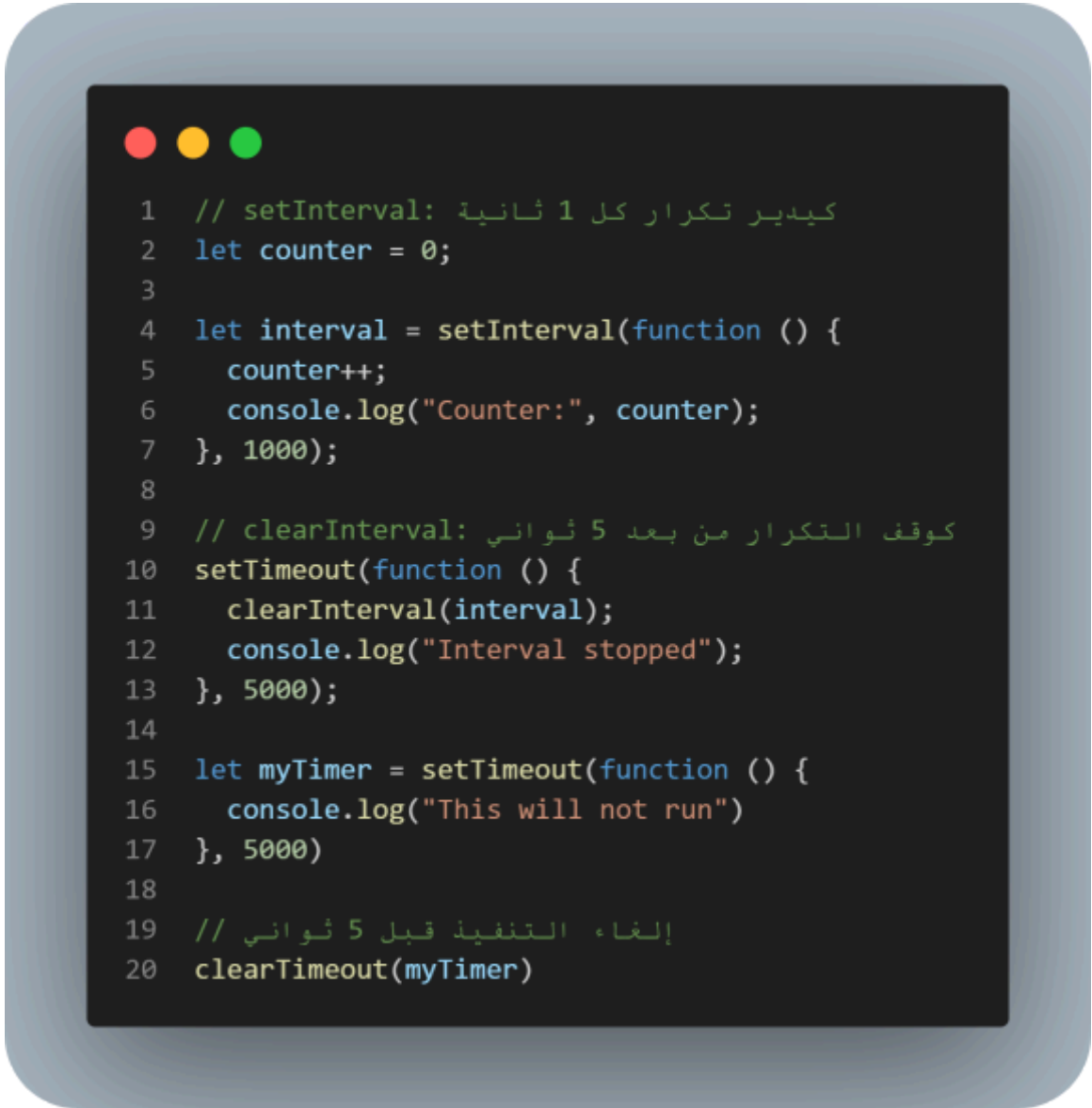


```
1  /* =====
2      clearTimeout()
3      =====
4      قبل ما يتنفذ setTimeout كتوقف
5  */
6
7  let myTimer = setTimeout(function () {
8      console.log("This will not run")
9  }, 5000)
10
11  // إلغاء التنفيذ قبل 5 ثواني
12  clearTimeout(myTimer)
```

2-2-setInterval and clearInterval:

setInterval: is a function that repeatedly runs code at a fixed time interval.

clearInterval: stops a running setInterval.



```
1 // setInterval: كيدير تكرار كل 1 ثانية
2 let counter = 0;
3
4 let interval = setInterval(function () {
5     counter++;
6     console.log("Counter:", counter);
7 }, 1000);
8
9 // clearInterval: كوقف التكرار من بعد 5 ثواني
10 setTimeout(function () {
11     clearInterval(interval);
12     console.log("Interval stopped");
13 }, 5000);
14
15 let myTimer = setTimeout(function () {
16     console.log("This will not run")
17 }, 5000)
18
19 // إلغاء التنفيذ قبل 5 ثواني
20 clearTimeout(myTimer)
```


2-3-Window Methods:

window.open(): is a function used to open a new browser window or tab with custom URL and features.

Window.close(): is a function used to close the current browser window, usually one opened with JavaScript.



```
1  // نافذة جديدة
2  let myWindow = window.open(
3      "https://google.com", // الرابط
4      "_blank",             // جديد tab فتح في
5      "width=600,height=400,left=200,top=100"
6  );
7
8  // غلق النافذة
9  myWindow.close();
```

scrollTo: is a function used to scroll to a specific position in the page.

scrollBy: is a function used to scroll relative to the current position.

scrollTop: is a property that returns how many pixels the page has been scrolled vertically.

scrollX: is a property that returns how many pixels the page has been scrolled horizontally.

```
1  /* =====
2      scrollBy()
3      =====
4      التحرك انطلاقاً من المكان الحالي
5  */
6
7  // 200px النزول من المكان الحالي
8  window.scrollTo({
9      top: 200,
10     left: 0,
11     behavior: "smooth"
12 });
13
14 /* =====
15     scrollY
16     =====
17     معرفة مكان النزول العمودي
18 */
19 console.log(window.scrollY);
20
21 /* =====
22     scrollX
23     =====
24     معرفة مكان الحركة الأفقية
25 */
26 console.log(window.scrollX);
27
28 /* =====
29     Example Real
30     =====
31 */
32
33 // مراقبة scroll
34 window.addEventListener("scroll", function () {
35
36     // طباعة مكان المستخدم في الصفحة
37     console.log("Y:", window.scrollY);
38     console.log("X:", window.scrollX);
39 });
```

3-Local Storage:

is a way to store data in the browser permanently, even after closing the page or browser We use it to save:

- theme (dark/light)
- language
- user settings

Methods:

```
1  /* =====
2    setItem()
3    =====
4    تخزين البيانات داخل localStorage
5  */
6
7  localStorage.setItem("name", "Hatim");
8  localStorage.setItem("age", "18");
9  localStorage.setItem("color", "red");
10
11 /* =====
12    getItem()
13    =====
14    جلب البيانات من localStorage ب مفتاح
15  */
16
17 console.log(
18     localStorage.getItem("name")
19 );
20 console.log(
21     localStorage.getItem("age")
22 );
23 console.log(
24     localStorage.getItem("color")
25 );
```



```
1  /* =====
2      removeItem()
3      =====
4      حذف عنصر واحد فقط
5  */
6  localStorage.removeItem("age");
7
8  /* =====
9      clear()
10     =====
11     حذف جميع البيانات
12 */
13 // localStorage.clear();
14
15 /* =====
16     length
17     =====
18     معرفة عدد العناصر المخزنة
19 */
20
21 console.log(localStorage.length);
22
23 /* =====
24     key()
25     =====
26     index حسب key جلب اسم
27 */
28
29 console.log(localStorage.key(0));
30
```

4-Session Storage:

is a way to store data temporarily in the browser, and it is cleared when the tab or browser is closed.

```
1  /* =====
2    setItem()
3    =====
4    تخزين البيانات
5  */
6  sessionStorage.setItem("name", "Hatim");
7
8  /* =====
9    getItem()
10   =====
11   جلب البيانات
12  */
13  console.log(
14    sessionStorage.getItem("name")
15  );
16
17  /* =====
18    removeItem()
19    =====
20    حذف عنصر واحد
21  */
22  sessionStorage.removeItem("name");
```



```
1  /* =====
2    clear()
3    =====
4    حذف جميع البيانات
5  */
6  // sessionStorage.clear();
7
8  /* =====
9    length
10   =====
11   عدد العناصر المخزنة
12  */
13  console.log(sessionStorage.length);
14
15  /* =====
16    key()
17   =====
18   index حسب key جلب اسم
19  */
20  console.log(sessionStorage.key(0));
```

5-Destructuring:

is a way to extract values from arrays or objects and store them directly into variables.

Array Example:

```
1 // Without Destructuring
2 let colors = ["red", "green", "blue"];
3 let first = colors[0];
4 let second = colors[1];
5
6 console.log(first);
7 console.log(second);
8
9 // Destructuring
10 // استخراج القيم مباشرة
11 let colors = ["red", "green", "blue"];
12 let [first, second, third] = colors;
13
14 console.log(first);
15 console.log(second);
16 console.log(third);
17
18 // Advanced Example
19 let myFriends1 = ["Ahmed", "Sayed", "Ali", ["Shady", "Amr", ["Mohamed", "Gamal"]]];
20 let [, , , [a1, , [, b2]]] = myFriends1;
21 console.log(a1); // Shady
22 console.log(b2); // Gamal
```

Object Example:

```
1  const user1 = {
2    theName: "Osama",
3    theAge: 39,
4    theTitle: "Developer",
5    theCountry: "Egypt",
6  };
7
8  /* =====
9    Object Destructuring
10   =====
11
12   object استخراج القيم من
13   variables وتخزينها مباشرة في
14  */
15
16  const {
17    // يأخذ قيمة theName
18    theName: t,
19    // يأخذ قيمة theAge
20    theAge,
21    // يأخذ قيمة theCountry
22    theCountry
23  } = user1;
24
25
26  /* =====
27    طباعة القيم
28   =====
29  */
30  console.log(t); // Osama
31  console.log(theAge); // 39
32  console.log(theCountry); // Egypt
```


Mixed Example:

```
1  const user9 = {
2    theName: "Osama",
3    theAge: 39,
4    // Array داخل object
5    skills: ["HTML", "CSS", "JavaScript"],
6    // Object داخل object
7    addresses: {
8      egypt: "Cairo",
9      ksa: "Riyadh",
10   },
11 };
12
13 /* =====
14    Object Destructuring
15    =====
16   */
17 const {
18   /*
19    variable تغيير اسم
20    theName => n1
21   */
22   theName: n1,
23   /*
24    theAge => a9
25   */
26   theAge: a9,
27   /*
28    skills array الدخول إلى
29
30    , three
31    => تجامل أول عنصر
32    => تجامل ثاني عنصر
33    => أخت ثالث عنصر
34   */
35   skills: [, , three],
36
37   /*
38    addresses object الدخول إلى
39    egypt => e2
40   */
41   addresses: {
42     egypt: e2
43   },
44 } = user9;
```

6-Regular Expression(Regex):

is a pattern used to search, validate, or manipulate text We use it to:Search within texts, Check email or password....

6-1-Rules Regex:

```
1  /* =====
2      إنشاء Regex
3      =====
4  */
5  // الطريقة الأولى
6  let reg1 = /hatim/;
7
8  // الطريقة الثانية
9  let reg2 = new RegExp("hatim");
10
11 /* =====
12     test()
13     =====
14     موجود pattern التحقق هل
15 */
16 console.log(/hatim/.test("hello hatim")); // true
17
18 /* =====
19     Flags
20     =====
21 */
22 // i = تجاهل الحروف الكبيرة والصغيرة
23 console.log(/hatim/i.test("HATIM")); // true
24 // g = البحث عن جميع النتائج
25 console.log("hatim hatim".match(/hatim/g));
```

6-1-1-Characters:

```
1  /* =====
2     Dot .
3     =====
4     واحد character أي
5  */
6  console.log(/h.t/.test("hat")); // true
7  console.log(/h.t/.test("h7t")); // true
8
9  /* =====
10     \d
11     =====
12     أي رقم
13  */
14  console.log(/\d/.test("5")); // true
15
16  /* =====
17     \D
18     =====
19     أي شيء ليس رقم
20  */
21  console.log(/\D/.test("A")); // true
22
23  /* =====
24     \w
25     =====
26     _ حرف أو رقم أو
27  */
28  console.log(/\w/.test("_")); // true
29
30  /* =====
31     \W
32     =====
33     _ أي شيء ليس حرف أو رقم أو
34  */
35  console.log(/\W/.test("@")); // true
36
37  /* =====
38     \s
39     =====
40     space
41  */
42  console.log(/\s/.test(" ")); // true
43
44  /* =====
45     \S
46     =====
47     أي شيء ليس space
48  */
49  console.log(/\S/.test("A")); // true
```

6-1-2-Quantifiers:

```
1  /* =====
2      +
3      =====
4      واحد أو أكثر
5  */
6  console.log(/a+/.test("aaaa")); // true
7
8  /* =====
9      *
10     =====
11     صفر أو أكثر
12 */
13 console.log(/a*/.test("")); // true
14
15 /* =====
16     ?
17     =====
18     صفر أو واحد
19 */
20 console.log(/colou?r/.test("color")); // true
21 console.log(/colou?r/.test("colour")); // true
22
23 /* =====
24     {n}
25     =====
26     عدد محدد
27 */
28 console.log(/\d{3}/.test("123")); // true
29
30 /* =====
31     {n,m}
32     =====
33     من n إلى m
34 */
35 console.log(/\d{2,5}/.test("1234")); // true
```

6-1-3-Position:

```
1  /* =====
2     ^
3     =====
4     بداية النص
5  */
6  console.log(/^hatim/.test("hatim js")); // true
7
8  /* =====
9     $
10    =====
11    نهاية النص
12  */
13 console.log(/js$/.test("hatim js")); // true
```

6-1-4-Groups And Ranges:

```
1  /* =====
2     []
3     =====
4     مجموعة حروف
5  */
6  console.log(/[abc]/.test("a")); // true
7
8  /* =====
9     [^]
10    =====
11    نفى
12  */
13  console.log(/^[abc]/.test("z")); // true
14
15  /* =====
16     Range
17     =====
18  */
19  console.log(/[a-z]/.test("h")); // true
20  console.log(/[0-9]/.test("5")); // true
21
22  /* =====
23     |
24     =====
25     OR
26  */
27  console.log(/html|css|js/.test("css")); // true
```

7-Date, Generator, Modules:

7-1-Date:

is an object used to work with:

- dates
- time
- days
- hours
- minutes
- seconds

It is used to:

- get the current date and time
- calculate differences between dates
- create timers
- display dates in websites and applications

7-1-1-get Date And Time:

They are methods used to get the current date and time from the **Date()** object in JavaScript such as year, month, day, hours, etc.



```
1  /*
2  getTime()
3  milliseconds ترجع عدد
4  حتى الآن January 1970 من 1
5  */
6  console.log(dateNow.getTime());
7
8  /*
9  getDate()
10  ترجع يوم الشهر الحالي
11  من 1 حتى 31
12  */
13  console.log(dateNow.getDate());
14
15  /*
16  getFullYear()
17  ترجع السنة الحالية كاملة
18  */
19  console.log(dateNow.getFullYear());
20
21  /*
22  getMonth()
23  ترجع رقم الشهر
24  لكن:
25  الشهور تبدأ من 0
26  يعني:
27  0 = January
28  1 = February
29  ...
30  11 = December
31  */
32  console.log(dateNow.getMonth());
```




```
1  /*
2  getDay()
3  ترجع رقم اليوم في الأسبوع
4  0 = Sunday
5  1 = Monday
6  ...
7  6 = Saturday
8  */
9  console.log(dateNow.getDay());
10
11 /*
12 getHours()
13 ترجع الساعة الحالية
14 */
15 console.log(dateNow.getHours());
16
17 /*
18 getMinutes()
19 ترجع الدقائق الحالية
20 */
21 console.log(dateNow.getMinutes());
22
23 /*
24 getSeconds()
25 ترجع الثواني الحالية
26 */
27 console.log(dateNow.getSeconds());
```

7-1-2-Set Date And Time:

They are methods used to modify or set date and time values inside a **Date()** object such as year, month, day, hours, etc.

```
1  // إنشاء تاريخ جديد
2  let dateNow = new Date();
3
4  /* =====
5     setFullYear()
6     =====
7     تغيير السنة
8  */
9  dateNow.setFullYear(2030);
10
11 /* =====
12    setMonth()
13    =====
14    تغيير الشهر
15
16    الشهور تبدأ من 0
17    0 = January
18 */
19 dateNow.setMonth(5);
20
21 /* =====
22    setDate()
23    =====
24    تغيير يوم الشهر
25 */
26 dateNow.setDate(15);
```



```
1  /* =====
2      setHours()
3      =====
4      تغيير الساعة
5  */
6  dateNow.setHours(10);
7
8  /* =====
9      setMinutes()
10     =====
11     تغيير الدقائق
12 */
13 dateNow.setMinutes(30);
14
15 /* =====
16     setSeconds()
17     =====
18     تغيير الثواني
19 */
20 dateNow.setSeconds(50);
```

7-2-Generators:

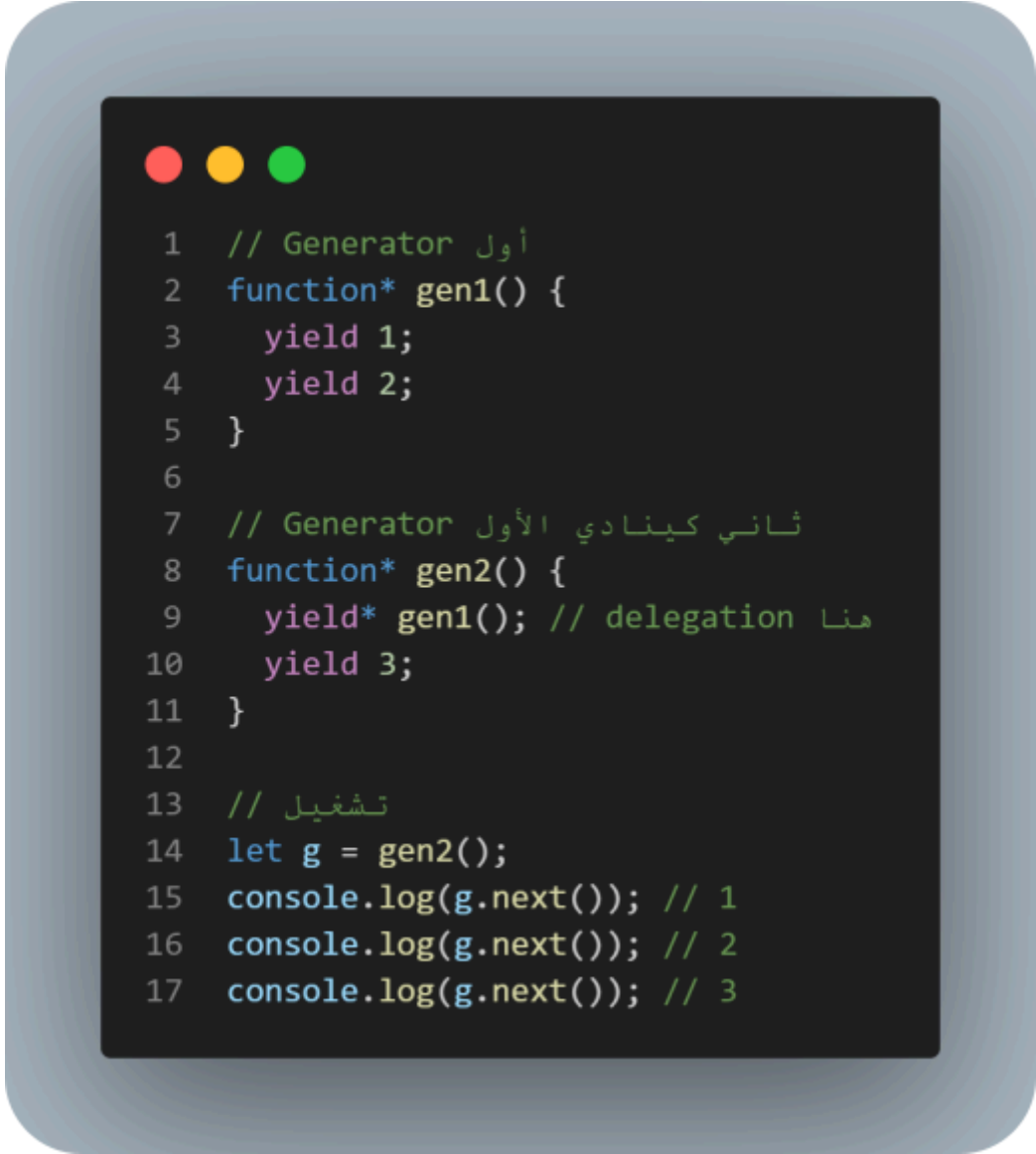
are special functions that can pause execution and return values, then resume later from the same point.

```
1 function* generateNumber() {
2   // كتموصل هنا next() أول مرة
3   yield 1;
4   /*
5    ملاحظة مهمة:
6    التنفيذ سيتوقف عند yield 1 من بعد
7    مرة أخرى next() حتى ندير console.log وما كيتنفذش
8   */
9   console.log("hello after yield 1");
10  // next() كتموصل هنا الثانية
11  yield 2;
12  // next() الثالثة
13  yield 3;
14  // next() الرابعة
15  yield 4;
16 }
17 // تشغيل generator
18 let Generator = generateNumber();
19 /* =====
20  وتكمل من نفس المكان value كترجع next() كل
21  =====
22 */
23 console.log(Generator.next());
24 // { value: 1, done: false }
25 console.log(Generator.next());
26 // هنا غادي يطبع "hello after yield 1"
27 // { value: 2, done: false }
28 console.log(Generator.next());
29 // { value: 3, done: false }
30 console.log(Generator.next());
31 // { value: 4, done: false }
32 console.log(Generator.next());
33 // { value: undefined, done: true }
```

7-2-1-Generator Delegate:

yield* allows one generator to delegate its execution to another generator.

Example:



```
1 // Generator أول
2 function* gen1() {
3   yield 1;
4   yield 2;
5 }
6
7 // Generator الثاني كينادي الأول
8 function* gen2() {
9   yield* gen1(); // delegation هنا
10  yield 3;
11 }
12
13 // تشغيل
14 let g = gen2();
15 console.log(g.next()); // 1
16 console.log(g.next()); // 2
17 console.log(g.next()); // 3
```

7-3-Modules:

Modules are a way to split code into separate files and reuse them in different parts of a program.

7-3-1-Export:



```
1  // file: math.js
2  export const sum = (a, b) => {
3    return a + b;
4  };
5
6  export const mul = (a, b) => {
7    return a * b;
8  };
```

7-3-2-Import:



```
1 // file: main.js
2 import { sum, mul } from "./math.js";
3
4 console.log(sum(2, 3)); // 5
5 console.log(mul(2, 3)); // 6
```

7-3-3-Import All:



```
1 // file: main.js
2 import * as math from "./math.js";
3
4 console.log(math.sum(2, 3));
5 console.log(math.mul(2, 3));
```

>API (Application Program Interface):

is a way for two systems or applications to communicate and exchange data.

We Use API For:

- get data from the internet
- send data to a server
- do login / signup

1-JSON (JavaScript Object Notation):

is a lightweight data format used to store and exchange data between a server and applications.

We use JSON For:

- Easy To Use And Read.
- Used By Most Programming Languages And Its Frameworks.
- You Can Convert JSON Object To JS Object And Vice Versa.

2-Object VS JSON:

What is the difference between object and JSON?

Example:

```
1  /* =====
2    1) Object
3    =====
4  */
5  let user = {
6    name: "Hatim",
7    age: 18,
8    city: "Tangier"
9  };
10 /*
11 Object:
12 - هذا كود داخل JavaScript
13 - يمكن نستعملو مباشرة
14 */
15
16 /* =====
17    2) JSON (string format)
18    =====
19 */
20 /*
21 JSON:
22 - هو نفس البيانات ولكن على شكل STRING
23 - خاص يكون بين ""
24 */
25 let jsonData = '{"name":"Hatim","age":18,"city":"Tangier"}';
```



```
1  /* =====
2      3) تحويل JSON → Object
3      =====
4  */
5  let obj = JSON.parse(jsonData);
6
7  /*
8      عادي Object دابا رجعنا
9      object يمكن نستعملو بحال أي
10 */
11 console.log("AFTER PARSE:", obj);
12 console.log(obj.name);
13
14 /* =====
15      4) تحويل Object → JSON
16      =====
17 */
18 let backToJSON = JSON.stringify(user);
19
20 /*
21      JSON string إلى object تحويل
22 */
23 console.log("STRINGIFY:", backToJSON);
```

3-Synchronous VS ASynchronous:

Synchronous: Code executes line by line, and each line must finish before the next starts.

Asynchronous: Code can run tasks in parallel without waiting for others to finish.

```
1  /* =====
2     Synchronous (متزامن)
3     =====
4  */
5  console.log("1"); // كيتنفذ مباشرة
6  console.log("2"); // كيتنفذ من بعدو مباشرة
7  console.log("3"); // كيتنفذ من بعد
8  /*
9  النتيجة:
10 1
11 2
12 3
13 كتنفذ سطر بـ سطر JavaScript حيث
14 */
15
16 /* =====
17     Asynchronous (غير متزامن)
18     =====
19  */
20 console.log("1"); // كيتنفذ مباشرة
21 setTimeout(() => {
22     console.log("2");
23     // هاد المظرف غادي يتأخر 2 ثواني
24 }, 2000);
25 console.log("3"); // كيتنفذ مباشرة ما كيتنظرش
26
27 /*
28 النتيجة:
29 1
30 3
31 2
32 Web API كيدخل في setTimeout حيث
33 ما كيتسنا موش JavaScript و
34 */
```

4-AJAX:

is a technique that allows sending and receiving data from a server without reloading the whole page.

Example AJAX:

```
1  /* =====
2      1) إنشاء Request
3      =====
4  */
5  let Myrequest = new XMLHttpRequest();
6
7  /* =====
8      2) فتح الاتصال مع API
9      =====
10     GET = نجيب data
11     repos كيرجع GitHub API الرابط ديال
12  */
13  Myrequest.open(
14      "GET",
15      "https://api.github.com/users/HatimElbakkali/repos",
16      true // async = true (ما نوقفش الصفحة)
17  );
18
19  /* =====
20      3) إرسال الطلب للسيرفر
21      =====
22  */
23  Myrequest.send();
```



```
1  /* =====
2     مراقبة حالة الطلب (4)
3     =====
4  */
5  Myrequest.onreadystatechange = function () {
6      /*
7         readyState === 4
8         يعني: الطلب تسالي وجانا الرد
9
10        status === 200
11        (OK) يعني: كلشي مزيان
12    */
13    if (this.readyState === 4 && this.status === 200) {
14        /* =====
15           (JSON string) البيانات جاية كنص (5)
16           =====
17        */
18        console.log(this.responseText);
19
20        /* =====
21           JSON → Object تحويل (6)
22           =====
23        */
24        let jsData = JSON.parse(this.responseText);
25
26        /* =====
27           repos على loop (7)
28           =====
29        */
30        for (let i = 0; i < jsData.length; i++) {
31            /* =====
32               HTML إنشاء عنصر (8)
33               =====
34            */
35            let div = document.createElement("div");
36
```

```
1      /* =====
2      9) جلب اسم repo
3      =====
4      */
5      let repoName = document.createTextNode(jsData[i].full_name);
6
7      /* =====
8      10) إدخال الاسم داخل div
9      =====
10     */
11     div.append(repoName);
12
13     /* =====
14     11) إضافة div للصفحة
15     =====
16     */
17     document.body.append(div);
18 }
19 }
20 };
```

Clarification:

ReadState: A property that indicates the current state of the request.

0 => Request Not Initialized

1 => Server Connection Established

2 => Request Received

3 => Processing Request

4 => Request Finished And Response Is Ready

Status: A property that indicates whether the request succeeded or failed.

200 => OK

201 => Created

404 => Not Found

500 => Server Error

New XMLHttpRequest(): Creates a new XMLHttpRequest object used to send and receive data from a server.

Onreadystatechange: An event that runs whenever the request state change.

5-Promise:

is an object that represents the eventual success or failure of an asynchronous operation.

```
1  /* =====
2   إنشاء Promise
3   =====
4   فيها function كاستقبال
5   جوج parameters:
6   resolve => تستعملها على العملية تنجح
7   reject  => تستعملها على العملية تفشل
8  */
9
10 let myPromise = new Promise(function (resolve, reject) {
11   /* =====
12    متغير كيمثل نتيجة العملية
13    =====
14    true  => نجاح
15    false => فشل
16   */
17   let success = true;
18
19   /* =====
20    التحقق من النتيجة
21    =====
22   */
23   if (success) {
24     /*
25      resolve()
26      نجاحات Promise كتعني أن
27      والقيمة التي داخلها
28      then() غادي تعطي لـ
29     */
30     resolve("Operation Success");
31   } else {
32     /*
33      reject()
34      فشلات Promise كتعني أن
35      والقيمة التي داخلها
36      catch() غادي تعطي لـ
37     */
38     reject("Operation Failed");
39   }
40 }
41 );
42 ));
```


5-1-then, catch, finally:

Then: Runs when the Promise is successfully resolved.

Catch: Runs when the Promise is rejected (error).

Finally: Runs always, whether the Promise is resolved or rejected.

```
1 // متغير كيمثل واث العملية نجاحات ولا فشلات
2 let success = true;
3
4 // جديدة Promise إنشاء
5 let myPromise = new Promise((resolve, reject) => {
6   // إذا كانت العملية ناجحة
7   if (success) {
8     // resolve = نجاح Promise
9     // القيمة "Operation Success"
10    // غادي تعطي لـ then()
11    resolve("Operation Success");
12  }
13 } else {
14   // reject = فشل Promise
15   // القيمة "Operation Failed"
16   // غادي تعطي لـ catch()
17   reject("Operation Failed");
18 }
19 });
20
21 // التعامل مع Promise
22 myPromise
23   // (resolve) ناجحة Promise كتحكم إذا كانت
24   .then((result) => {
25     // result = القيمة التي جات من resolve()
26     console.log("Then:", result);
27   })
28   // (reject) فاشلة Promise كتحكم إذا كانت
29   .catch((error) => {
30     // error = القيمة التي جات من reject()
31     console.log("Catch:", error);
32   })
33   // Promise كتحكم دائما سواء نجاحات أو فشلات
34   .finally(() => {
35     console.log("Finally: Always runs");
36   })
37   .finally();
38 }
```

5-2-Promise.all, Promise.allSettled, Promise.race :

Promise.all: returns a single result when all Promises are fulfilled. If any Promise is rejected, it immediately rejects.

Promise.allSettled: returns the result of all Promises, showing whether each one is fulfilled or rejected.

Promise.race: returns the first completed Promise among many Promises.

```
1  let p1 = new Promise((resolve) => {
2    setTimeout(() => {
3      resolve("User Data");
4    }, 3000);
5  });
6
7  let p2 = new Promise((resolve) => {
8    setTimeout(() => {
9      resolve("Posts Data");
10   }, 1000);
11 });
12
13 let p3 = new Promise((resolve, reject) => {
14   setTimeout(() => {
15     reject("Comments Error");
16   }, 2000);
17 });
18
19
20 // =====
21 // 1) Promise.all()
22 // =====
23 /*
24  - پنجو Promises کیننظر جمیع
25  - لا واحد فشل → کاملین یفشلو -
26  */
27 Promise.all([p1, p2, p3])
28   .then((result) => {
29     console.log("ALL:", result);
30   })
31   .catch((error) => {
32     console.log("ALL ERROR:", error);
33   });
```



```
1 // =====
2 // 2) Promise.allSettled()
3 // =====
4 /*
5  - يكملو Promises كينتظر جميع
6  - كيعطيك نتيجة كل واحد (نجاح أو فشل)
7  */
8 Promise.allSettled([p1, p2, p3])
9   .then((result) => {
10     console.log("ALL SETTLED:", result);
11   });
12
13
14 // =====
15 // 3) Promise.race()
16 // =====
17 /*
18  - يسالي Promise كيخرج أول
19  - ما كيهتمش واش نجاح أو فشل
20  */
21 Promise.race([p1, p2, p3])
22   .then((result) => {
23     console.log("RACE:", result);
24   })
25   .catch((error) => {
26     console.log("RACE ERROR:", error);
27   });
28
```

6-Fetch API:

is the modern way to send and receive data from a server.

Example Get Data:

```
1  fetch("https://api.github.com/users/octocat")
2    // تحويل الرد من JSON إلى JavaScript Object
3    .then((response) => response.json())
4
5    // استعمال البيانات
6    .then((data) => {
7      console.log(data.login);
8      console.log(data.id);
9    })
10
11   // معالجة الأخطاء
12   .catch((error) => {
13     console.log(error);
14   })
15
16   // يُنفذ دائماً سواء نجح الطلب أو فشل
17   .finally(() => {
18     console.log("انتهى الطلب");
19   });
```

Example Send Data:



```
1 fetch("https://api.example.com/users", {
2   /* =====
3   نوع الطلب
4   =====
5   POST = إرسال data للميرفر
6   */
7   method: "POST",
8
9   /* =====
10  معلومات على البيانات
11  =====
12  Content-Type: application/json
13  JSON غادي تعني بصيغة data يعني
14  */
15  headers: {
16    "Content-Type": "application/json"
17  },
18
19  /* =====
20  البيانات التي غادي نرسلو
21  =====
22  JSON.stringify لذلك كنستعملو String خاصها تكون
23  */
24  body: JSON.stringify({
25    name: "Hatim",
26    age: 20
27  })
28 })
29
30 /* =====
31 ديال الميرفر
32 =====
33 Object إلى JSON كنحولوه من
34 */
35 .then((response) => response.json())
36
37 /* =====
38 النتيجة النهائية
39 =====
40 هنا كنستعملو البيانات التي رجعات من الميرفر
41 */
42 .then((data) => {
43   console.log("Success:", data);
44 })
45
46 /* =====
47 معالجة الأخطاء
48 =====
49 (internet / server error) ولا وقع مشكل
50 */
51 .catch((error) => {
52   console.log("Error:", error);
53 });
```

7-Async/Await, Try:

Async: used before a function to make it return a Promise and allow the use of **await**.

Await: used to wait for a Promise to finish and return its result.

Try: is used to test a block of code that might cause an error.

Example:



```
1  try {  
2    // نحاولو ننفذو هاد الكود  
3    // error ما معرفاش → غادي يوقع x ولكن  
4    console.log(x);  
5  } catch (error) {  
6    // try إلا وقع أي خطأ داخل  
7    // البرنامج ما كيتوقفش  
8    // وكنمشي هنا باش نعالجو المشكل  
9    console.log("Error happened");  
10 }
```

Full Example:



```
1  async function fetchData() {
2    // كبتطيع أول حاجة
3    console.log("Before Fetch");
4    try {
5      /* =====
6        إرسال Request إلى GitHub API
7        =====
8        Response كتمنى حتى يوصل
9      */
10     let myData1 = await fetch(
11       "https://api.github.com/users/HatimElbakkali/repos"
12     );
13
14     /* =====
15       تحويل JSON إلى JS Object
16       =====
17       Promise كترجع json()
18       await لذلك استعملنا
19     */
20     console.log(await myData1.json());
21   } catch (reason) {
22     /* =====
23       معالجة الأخطاء
24       =====
25       fetch إذا وقع خطأ في
26       json() أو
27       معلومات الخطأ reason =
28     */
29     console.log(`Reason: ${reason}`);
30
31   } finally {
32     /* =====
33       يتنفيذ دائماً
34       =====
35       سواء نجح الطلب أو فشل
36     */
37     console.log("After Fetch");
38   }
39 }
40 // استدعاء الدالة
41 fetchData();
```

**Thanks
for
Reading
PDF
JavaScript**